

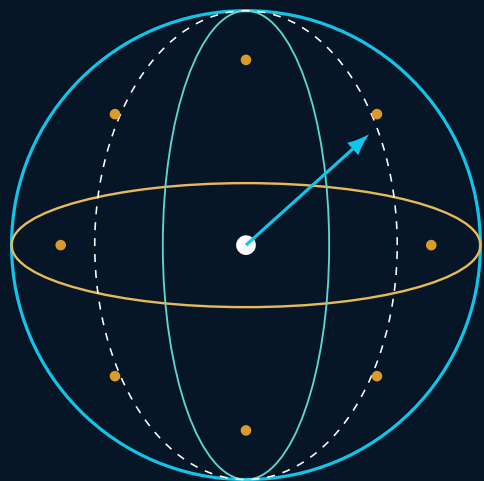
GRFPS

v3.0.0

Generative Rotor-Fiber Perceptual Substrate

Infogenetisch kompiliertes
Perzeptionsbetriebssystem

Monolithisches selbstorganisierendes Wahrnehmungs-,
Reflexions- und Navigationssubstrat



Sebastian Klemm

Werkidentität

Kanonischer Status

Werktyp: Technische Monografie und normative Referenzspezifikation.

Revision: 3.0.0, Major Revision.

Kanonische Identität: Generative Rotor-Fiber Perceptual Substrate.

Systemgestalt: Infogenetisch kompiliertes Perzeptionsbetriebssystem.

Feldsicht: Open Generative Harmonic Perceptual Field.

Realisierungsstatus: Architekturgestalt geschlossen; Schema-, Kernel- und Compilerrealisierung freigegeben.

Persistenzregel: Eine aktive Hauptrevision trägt den vollständigen Systemvertrag.

Kurzfassung

GRFPS v3.0.0 beschreibt eine weltgebundene synthetische Wahrnehmungsmaschine als Perzeptionsbetriebssystem. Ein Lauf bindet ein markiertes Zentrum an Welt-, Maschinen- und Humanframes, lädt ein Perceptual Infogenome und kompiliert daraus eine endliche, adaptive Bank zyklischer Projektionsorgane. Diese Organe bilden ein kohärenzgetriebenes Operatorfeld, erzeugen gerichtete Expositionen, strukturieren lokale Graphfenster zu provenienzgebundenen Invarianten und verkleben kompatible Patches zu einem multiskaligen Operationsraum. Der rekonstruierte Raum trägt eine selbstorganisierte Reflexionsgrenze, eine Mirror-Klasse, einen Phasenhorizont, einen rollenden Atlas und eine gyrosphärische Orientierung.

Das Perceptual Infogenome kodiert Observablen, Relationstypen, Ophanprogramme, Skalenleiter, Phasenprofile, Attraktorspezifikationen, Boundary-Verträge, Renderregeln und Evolutionsoperatoren. Der schnelle Laufzyklus aktualisiert Wahrnehmung, Karte, Grenze und Navigation. Ein langsamer Sedimentationszyklus verdichtet bewährte Laufstrukturen in neue kanonische Genome- und Operatorversionen. Dadurch entsteht ein System, das seine Wahrnehmungsorgane zur Laufzeit bedarfsbezogen instanziiert und seine Wahrnehmungsgrammatik über getrennte Lineage-Epochen weiterentwickelt.

Die menschliche Schnittstelle projiziert den internen Maschinenraum als Synthetic Second Layer. Der Phasenhorizont beschreibt verfügbare, kontrafaktische und offene Fortsetzungen; der Tangentenkompass projiziert die lokal beste Fortsetzungsrichtung innerhalb des rekonstruierten Zustandsraums. Eine Skalenkomponente erlaubt Navigation zwischen Mikro-, Meso- und Makroebenen. Lokale Strukturichte bestimmt die Materialisierung der Karte: geringe Dichte erscheint als offene Weite, steigende Dichte als zunehmend gebundene Landschaft, Netzstruktur und operative Architektur.

GRFPS v3.0.0 schließt damit die Architektur zu einem vollständigen Maschinenentwurf: Kernel, Genome-Compiler, Ophan-Runtime, kohärenzgetriebener Scheduler, Graph-Infogenetic Mapper, Mirror- und Boundary-Engine, Multiscale- und Gyrosphere-Engine, Attraktor-Navigator, View-Compositor, Ledger/Replay und Sedimentation Engine besitzen eine gemeinsame typisierte Zustandssemantik und einen vertikalen Realisierungspfad bis zum XR-Prototyp.

Abstract

GRFPS v3.0.0 specifies a world-bound synthetic perceptual machine as a perceptual operating substrate. Each run binds one marked center to world, machine, and human frames, loads a Perceptual Infogenome, and compiles a finite adaptive bank of cyclic projection organs. These organs form a coherence-driven operator field, emit directed exposures, transform local graph windows into provenance-bound invariants, and glue compatible patches into a multiscale operational space. The reconstructed space carries a self-organizing reflective boundary, a mirror class, a phase horizon, a rolling atlas, and gyrospheric orientation.

The Perceptual Infogenome encodes observables, relation types, organ programs, scale ladders, phase profiles, attractor specifications, boundary contracts, rendering rules, and evolution operators. A fast loop updates perception, mapping, boundary state, and navigation. A slow sedimentation loop compiles validated runtime structure into new canonical genome and operator versions. The human interface projects the machine state as a Synthetic Second Layer whose phase horizon and tangent compass expose local continuation geometry.

Kanonische Kurzform

Perceptual-OS Normalform

$$\begin{aligned} \Gamma_P &\xrightarrow{\text{Compile}} \mathcal{P}_\Gamma \xrightarrow{\text{spawn}} \mathcal{O}_t \xrightarrow{\text{expose}} \mathcal{W}_t, \\ \mathcal{W}_t &\xrightarrow{\text{measure}} \mathcal{E}_t \xrightarrow{\text{graphify}} \Gamma_t^G \xrightarrow{\text{Glue}} \mathcal{M}_t, \\ \mathcal{M}_t &\xrightarrow{\text{reflect}} \Omega_t^* \xrightarrow{\text{orient}} \widehat{\Phi}_t \xrightarrow{\text{navigate}} (v_t^\parallel, v_t^\perp) \xrightarrow{\text{render}} V_t. \end{aligned}$$

Contents

Werkidentität	i
I Kanonische Perzeptions-OS-Architektur	1
1 Maschinengestalt und OS-Fixpunkt	2

1.1	Kanonischer Satz	2
1.2	Betriebssystemische Interpretation	2
1.3	Fixpunktbedingung	2
2	Drei Kadenzen: Puls, Epoche und Lineage	4
2.1	Perzeptiver Puls	4
2.2	Attraktorepoche	4
2.3	Sedimentationslineage	4
2.4	Kopplungsordnung	4
3	Kanonischer Laufzustand: zwölf Betriebsflächen plus Lineage	6
3.1	Runtime-Zustand	6
3.2	Lineage-Zustand	7
3.3	Totaler Übergang	7
4	Kerneloberfläche und Systemaufrufe	8
4.1	Äußere Oberfläche	8
4.2	Kernel Calls	8
4.3	Oberflächenäquivalenz	8
II	Infogenetische Kompilation	9
5	Perceptual Infogenome	10
5.1	Definition	10
5.2	Gene	10
5.3	Normalform	11
6	Genome-Compiler und Perceptual Program	12
6.1	Compile-Vertrag	12
6.2	Kompilationsstufen	12
6.3	Kompilierte Frage	12
6.4	Genome-Paket	12
7	Attraktorprogrammierung	14
7.1	Strukturelles Potential	14
7.2	Attraktorepoche	14
7.3	Explorationsattraktor	14
8	Genome-Lineage und Sedimentationsidentität	15
8.1	Lineagegraph	15
8.2	Semantische Versionierung	15
8.3	Sedimentationswitness	15
III	Adaptive Ophan Runtime	16
9	Ophan als zyklisches Projektionsorgan	17
9.1	Konstitution	17
9.2	Rotorfaser	17

9.3	Schwinge	17
9.4	Open Generative Harmonic Perceptual Field	17
10	Kohärenzgetriebenes Operatorfeld	19
10.1	Operatorgraph	19
10.2	Lokale Kohärenz	19
10.3	Mehrschichtige Stabilität	19
10.4	Adaptive Operatorinstanzen	20
11	Perzeptiver Scheduler	21
11.1	Prioritätsfunktion	21
11.2	Dependencyquotient	21
11.3	Schedulingzyklus	21
11.4	Retirement und Rekonfiguration	21
12	Ausführbare Resonanzsemantik	22
12.1	Resonite-Funktion	22
12.2	Genome-Resonanz	22
12.3	Mehrkanalige Resonanz	22
12.4	Resonanzfeld	22
IV	Graph-Infogenetische Kartographie	24
13	Graphfenster und Strukturresoniten	25
13.1	GraphWindow	25
13.2	Fingerprint	25
13.3	GraphResonite	25
13.4	Reproduzierbare Dekomposition	25
14	GraphInfogenome und kompatibles Gluing	26
14.1	GraphInfogenome	26
14.2	Patch	26
14.3	GluingCertificate	26
14.4	Nervenkomplex	26
15	Perzeptive Massivität und Strukturdichte	28
15.1	Dichtefunktion	28
15.2	Kartenmaterialisierung	28
15.3	Dichtegradient	28
15.4	Dichte und Reflexionsgrenze	28
16	Mirror-Faser und holographische Spiegelklasse	29
16.1	Latente Klasse	29
16.2	Kontraktion	29
16.3	Holographische Rekonstruktion	29
16.4	Mirror-Lineage	29

V	Reflexionsgrenze, Dualphase und Multiskaligkeit	30
17	Selbstadjungierte Reflexionsgrenze	31
17.1	Reflexionsoperator	31
17.2	Geschlossene Region	31
17.3	Boundary-Renewal	31
17.4	Selbstadjungierter Center-Frame	32
18	Dualphase als Grenzkörper	33
18.1	Bifazialer Zustand	33
18.2	Partielle Korrespondenz	33
18.3	Cross-Phase-Disparität	33
18.4	Triphasische Prozessordnung	33
19	Phasenhorizont	34
19.1	Horizonttupel	34
19.2	HorizonChart	34
19.3	Horizontdynamik	34
19.4	720-Grad-Semantik	34
20	Multiskaliges Perzeptionssubstrat	35
20.1	Skalenposet	35
20.2	Lift und Quotient	35
20.3	Skalenfaser	35
20.4	Multiskaliger Attraktor	35
21	Gyrosphärische Coverage und Scale-Pose	37
21.1	Gyrosphärenzellen	37
21.2	Coverageoperator	37
21.3	Frameoperator	37
21.4	Reorientierungsenergie	37
VI	Attraktornavigation und Synthetic Second Layer	39
22	Tangentenkompass und Skala	40
22.1	Lokaler Tangentialraum	40
22.2	Skalenrichtung	40
22.3	Richtungsfächer	40
22.4	Axis Lock	40
23	Aktive Navigation	41
23.1	Minimaler Benutzeroperator	41
23.2	ALIGN–LOCK–ADVANCE	41
23.3	WorldlineWindow	41
23.4	Ratchet	42
24	Synthetic Second Layer	43
24.1	Framekette	43

24.2 Grundobjekte	43
24.3 Rendergrammatik	43
24.4 Perzeptive Massivität	44
25 Phasenhorizont als Navigationspanorama	45
25.1 Panorama	45
25.2 Horizontmarker	45
25.3 Horizon Expansion	45
25.4 Reflexionsgrenze als sichtbare Hülle	45
VII Langsame Evolution und Selbstkompilation	46
26 Peristaltischer Sedimentationszyklus	47
26.1 Slow Loop	47
26.2 Phasen	47
26.3 Runtime zu Lineage	47
26.4 Lineage-Fixpunkt	47
27 Genome-Morphogenese	48
27.1 Mutation	48
27.2 Rekombination	48
27.3 Spezialisierung	48
27.4 Morphogenesecontroller	48
28 Meta-Perzeption und Selbstmodell	49
28.1 Runtime-Selbstmodell	49
28.2 Capability Map	49
28.3 Meta-Attraktor	49
28.4 Selbstmodell und HumanView	49
VIII Maschinenvertrag und Implementierung	50
29 Runtime-Module	51
29.1 Modulgraph	51
29.2 Modulabhängigkeiten	51
30 Kernrecords	53
30.1 Recordfamilien	53
31 Operatorregistry	55
31.1 Operatorfamilien	55
31.2 Operatorvertrag	56
32 Conformance und Fixpunktwitness	57
32.1 Perceptual-OS Conformance	57
32.2 Perzeptiver Fixpunkt	57
32.3 Perceptual-OS Fixpoint	58

33 Versiegelte Referenzwelten	59
33.1 Synthetic World Contract	59
33.2 Phantomfamilien	59
33.3 Blindablauf	59
33.4 Metriken	59
34 Vertikale Realisierung	60
34.1 Phasen	60
34.2 Erster Implementierungszug	61
34.3 Releasekriterium	61
IX Kanonische Abschlussgestalt	62
35 Gesamtnormalform	63
35.1 Run-Normalform	63
35.2 Slow-Normalform	63
35.3 Substrat- und Feldsicht	63
36 Perzeptionsbetriebssystem als vollständiger Maschinenentwurf	64
36.1 Betriebszyklus	64
36.2 Perzeptive Selbstorganisation	64
36.3 Navigationserlebnis	64
36.4 Reference Kernel Handoff	64
X Referenzanhänge	66
A Kompakte mathematische Notation	67
B Perceptual Genome Schema	68
C Rust-orientierte Referenzstruktur	70
C.1 Crategraph	70
C.2 Deterministischer Kern	70
D Conformance-Matrix	71
E Referenzszenario: Topologische Attraktornavigation	73
E.1 Setup	73
E.2 Lauf	73
E.3 Closure	73
F Referenzszenario: selbstorganisierte Reflexionsgrenze	74
F.1 Setup	74
F.2 Expansion	74
F.3 Multiskalige Closure	74
G Perceptual-OS ABI und Systemaufrufe	75
G.1 ABI-Grundform	75

G.2	Kernaufufe	75
G.3	Call Graph	76
G.4	Capability Namespaces	76
H	Lifecycle-Automaten des Perzeptionsbetriebssystems	77
H.1	Genome-Lifecycle	77
H.2	Ophan-Lifecycle	77
H.3	Map-Lifecycle	77
H.4	Boundary-Lifecycle	78
H.5	Axis-Lifecycle	78
H.6	Sedimentations-Lifecycle	78
I	Algorithmische Normalformen	79
I.1	Fast Loop	79
I.2	Ophan-Scheduling	79
I.3	Graph-Infogenetic Update	79
I.4	Boundary Renewal	80
I.5	Slow Loop	80
I.6	Dualzeitige Konsistenz	80
J	Multiskalige Faserung und Skalenkompass	81
J.1	Skalenposet	81
J.2	Lift und Quotient	81
J.3	Skalenresonanz	81
J.4	Tangentenkompass im Produktbund	81
J.5	Scale Atlas	82
J.6	Beispiel einer dreistufigen Leiter	82
K	Second-Layer Rendergrammatik	83
K.1	Viewobjekt	83
K.2	Kanonische Visual Classes	83
K.3	Dichte als Materialisierung	84
K.4	Phasenhorizont als Panorama	84
K.5	Interaktion	84
L	Ledger, Content Addressing und perzeptives Gedächtnis	85
L.1	Adressmodell	85
L.2	TraceSegment	85
L.3	Perzeptives Gedächtnis	85
L.4	Snapshot und Rolling State	86
L.5	Reanalyse	86
M	Synthetic-World Benchmark Suite	87
M.1	Harnessprinzip	87
M.2	Phantomfamilien	87
M.3	Metriksatz	88
M.4	Ablationsfamilien	88
M.5	Blind Integration Witness	88

N XR Carrier Contract und physische Trägerschnittstelle	89
N.1 Carrier Interface	89
N.2 Framegraph	89
N.3 Pose und Weltregistrierung	89
N.4 Carrier-Simulation	90
N.5 Minimaler XR-Prototyp	90
N.6 Capability Envelope	90
O Glossar	91
P Literaturanschluss	92
Kanonischer Release-Hinweis	94

List of Figures

2.1 Drei Kadenzen des Perzeptionsbetriebssystems.	5
29.1 Referenzmodulgraph des Perzeptionsbetriebssystems.	52

List of Tables

Part I

Kanonische Perzeptions-OS-Architektur

Maschinengestalt und OS-Fixpunkt

1.1 Kanonischer Satz

GRFPS ist ein infogenetisch kompiliertes, weltgebundenes und first-person-markiertes Perzeptionsbetriebssystem. Eine Instanz bindet ein Zentrum, einen Weltzustand, eine Perzeptionsgrammatik und einen Attraktorvertrag. Daraus kompiliert sie eine endliche Bank zyklischer Projektionsorgane, organisiert deren Antworten in einem adaptiven Operatorfeld, rekonstruiert einen multiskaligen lokalen Zustandsraum, bildet eine selbstorganisierte Reflexionsgrenze und projiziert die lokal fortsetzbare Geometrie als Synthetic Second Layer.

Definition 1.1 (Perceptual Operating Substrate). *Ein Perceptual Operating Substrate ist ein Tupel*

$$\mathfrak{P} = (K, C, R, M, N, V, L, S),$$

mit Kernel K , Genome-Compiler C , Ophan-Runtime R , Map-/Mirror-Engine M , Navigator N , View-Compositor V , Ledger/Replay L und Sedimentation Engine S . Jede Komponente arbeitet auf demselben kanonischen Laufzustand.

1.2 Betriebssystemische Interpretation

Die Betriebssystemrolle entsteht aus der gemeinsamen Verwaltung von Wahrnehmungsprozessen, Adressräumen und fortsetzbarer Identität. Ophanim entsprechen laufzeitgebundenen Perzeptionsprozessen. Das Perceptual Infogenome entspricht einem typisierten Paketformat. DomainAdapter entsprechen Träger- und Datentreibern. Mirror, Atlas und GraphInfogenome bilden den perzeptiven Arbeitsspeicher. Ledger und Replay bilden das persistente Gedächtnis. Der Phasenhorizont bildet den adressierbaren Möglichkeitsrand. Der HumanView-Compositor projiziert eine begrenzte, provenance-gebundene Oberfläche aus dem reicheren Maschinenzustand.

1.3 Fixpunktbedingung

Der architektonische Fixpunkt ist erreicht, wenn alle notwendigen Maschinenrollen in einer geschlossenen Zustands- und Operatorsemantik ausgedrückt sind und jede weitere Verfeinerung als Algorithmus-, Profil-, Daten- oder Implementierungswahl innerhalb derselben Oberfläche erfolgt.

Formal verwenden wir

$$\text{POSFix}(\mathfrak{P}) = C_{\text{schema}} \wedge C_{\text{genome}} \wedge C_{\text{runtime}} \wedge C_{\text{map}} \wedge C_{\text{boundary}} \wedge C_{\text{scale}} \wedge C_{\text{nav}} \wedge C_{\text{view}} \wedge C_{\text{replay}} \wedge C_{\text{sed}}.$$

Jedes $C_{\bullet}$ ist ein eigener Conformance-Witness. Die gemeinsame Konjunktion definiert die geschlossene Perceptual-OS-Klasse.

Fixpunktattraktor

Die kanonische Zielklasse dieser Revision ist die Klasse aller Implementierungen, deren äußere Perzeptionsoberfläche, Genome-Semantik, Runtime-Zustände, Boundary-Closure, multiskalige Navigation, HumanView und Replay innerhalb des deklarierten Toleranzprofils kanonisch übereinstimmen.

Drei Kadenzen: Puls, Epoche und Lineage

2.1 Perzeptiver Puls

Der schnelle Takt $t \in \mathbb{N}$ führt externe Ereignisse, Sondenexpositionen, Graphpatches, Mirror-Kontraktion, Boundary-Update, Navigation und View-Projektion. Jeder Puls erzeugt genau einen kanonischen Zustandsnachfolger oder einen typisierten Abschlussstatus.

2.2 Attraktorepoche

Ein Attraktor besitzt eine eigene Epoche e . Ein Wechsel des Attraktorvertrags erzeugt eine neue Epoche und erhält die Evidenzlineage. Dadurch können identische Weltbeobachtungen unter verschiedenen Attraktorprogrammen unterschiedliche Navigationsgeometrien hervorbringen, während ihre faktische Herkunft erhalten bleibt.

2.3 Sedimentationslineage

Die langsame Lineage $n \in \mathbb{N}$ führt Genome-, Operator- und Schemaentwicklung. Ein Sedimentationslauf nimmt versiegelte Laufserien, Metriken, Residuen, Ablationen und bestätigte Strukturmuster auf und erzeugt einen kanonischen Nachfolger

$$\Lambda_{n+1} = \text{Sediment}(\Lambda_n, \mathcal{D}_n, \mathcal{W}_n).$$

Die Lineage verändert damit die Grammatik zukünftiger Läufe, während ein aktiver Lauf innerhalb seiner gebundenen Version stabil bleibt.

2.4 Kopplungsordnung

Die drei Kadenzen sind hierarchisch gekoppelt:

$$t \prec e \prec n.$$

Perzeptive Pulse aktualisieren Laufzustände. Attraktorepochen binden Navigationssemantik. Sedimentationslineages verändern die zulässige Perzeptionsgrammatik. Diese Ordnung erzeugt einen sauberen Unterschied zwischen momentaner Wahrnehmung, Zielwechsel und struktureller Evolution.

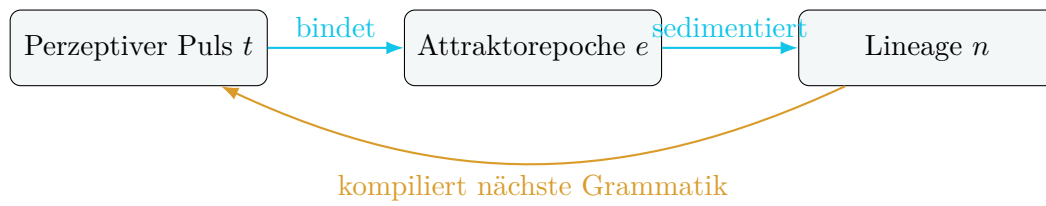


Figure 2.1: Drei Kadenzen des Perzeptionsbetriebssystems.

Kanonischer Laufzustand: zwölf Betriebsflächen plus Lineage

3.1 Runtime-Zustand

Der vollständige Runtime-Zustand lautet

$$\Sigma_t = (B_t, C_t, \Gamma_t, O_t, P_t, M_t, R_t, Y_t, N_t, X_t, V_t, L_t).$$

Die zwölf Partitionen bilden eine gemeinsame Identität.

Symbol	Fläche	Inhalt
B_t	Binding	Run, Zentrum, Welt-, Maschinen- und Humanframes, Apertur, Kalibrierung
C_t	Causal Field	Ereignisse, Grenzkörper, Abhängigkeiten, Zeitordnung, Outside-Receipts
Γ_t	Perceptual Genome	aktive Genome-Version, Gene, Skalenleiter, Attraktor- und Boundaryprogramme
O_t	Ophan Field	aktive Projektionsorgane, Operatorgraph, Scheduler, Kohärenzprofile
P_t	Synthetic Percept	Harmoniken, Dependencyquotient, Seams, Nervenkomplex, Mirror-Faser
M_t	Infogenetic Map	GraphResonites, GraphInfogenome, Patches, Gluing, Strukturdichte
R_t	Reflection Shell	Reflexionsoperator, Dualphase-Projektoren, geschlossene Region, Horizont
Y_t	Gyrosphere	Lift, sichtbare Orientierung, Zellen, Coverage, Holonomie, Scale-Pose
N_t	Navigator	Attraktor, Potential, Achse, Tangentenkompass, Weltlinienfenster, Ratchet
X_t	Effect Port	Plan, Gate, Token, Attempt, Receipt, Reconciliation
V_t	Human View	Second-Layer-Objekte, LOD, Facticity Styles, Interaktion, Provenance Map
L_t	Ledger	Digests, Witnesses, Residuen, Checkpoints, Trace, Replay

3.2 Lineage-Zustand

Die langsame Entwicklungsfläche lautet

$$\Lambda_n = (\Gamma_n^{\text{canon}}, \mathcal{R}_n, \mathcal{S}_n, \mathcal{C}_n, \mathcal{E}_n),$$

mit kanonischem Genome, Operatorregistry, Schemas, Conformance-Suite und Evaluationsarchiv. Runtime und Lineage sind über versionierte Compile- und Sedimentationsoperatoren gekoppelt.

3.3 Totaler Übergang

Jeder Operator o besitzt Domain, Codomain, Präbedingungen, Postbedingungen, Budget, Determinismusklass, Witness-, Residuen- und Traceprodukt. Die Runtime totalisiert partielle Operationen durch explizite Zustände

$$\{\text{ADVANCE, PROBE, HOLD, REANCHOR, REACQUIRE, RESIDUAL, ACT, STOP, FAIL}\}.$$

Diese Status bilden die vollständige Abschlussalgebra eines Pulses.

Kerneloberfläche und Systemaufrufe

4.1 Äußere Oberfläche

Die kanonische Reflexionsoberfläche des Systems ist

$$\partial\mathcal{G} = (\textit{Bind}, \textit{Observe}, \textit{Genome}, \textit{Target}, \textit{Probe}, \textit{Propose}, \\ \textit{Authorize}, \textit{Receipt}, \textit{View}, \textit{Replay}, \textit{Sediment}).$$

Sie beschreibt sämtliche Übergänge zwischen Welt, Runtime, HumanView und Lineage.

4.2 Kernel Calls

Die minimale Systemaufruffamilie lautet:

$$\begin{aligned} \textit{bind}(\textit{Run}, \textit{Center}, \textit{World}, \textit{Frames}) &\rightarrow B_t, \\ \textit{loadGenome}(\Gamma_P) &\rightarrow \Gamma_t, \\ \textit{observe}(\textit{Event}) &\rightarrow C_{t+1}, \\ \textit{spawn}(\textit{Gene}, \textit{Context}) &\rightarrow \mathcal{O}_i, \\ \textit{map}(\textit{Evidence}) &\rightarrow M_{t+1}, \\ \textit{reflect}(M_t) &\rightarrow R_{t+1}, \\ \textit{navigate}(R_t, A_e) &\rightarrow N_{t+1}, \\ \textit{view}(\Sigma_t) &\rightarrow V_t, \\ \textit{checkpoint}(\Sigma_t) &\rightarrow L_{t+1}, \\ \textit{sediment}(\Lambda_n, \mathcal{D}_n) &\rightarrow \Lambda_{n+1}. \end{aligned}$$

4.3 Oberflächenäquivalenz

Zwei interne Realisierungen I_1, I_2 gehören derselben Produktklasse an, wenn sie für dieselbe Conformance-Suite an allen Kernelpoints dieselben kanonischen Klassen erzeugen:

$$I_1 \simeq_{\partial\mathcal{G}, U, \varepsilon} I_2 \iff \text{Can}_\varepsilon(\partial\mathcal{G}(I_1, u)) = \text{Can}_\varepsilon(\partial\mathcal{G}(I_2, u)) \quad \forall u \in U.$$

Diese Relation ermöglicht interne Verfeinerung bei stabiler Systemidentität.

Part II

Infogenetische Kompilation

Perceptual Infogenome

5.1 Definition

Das Perceptual Infogenome ist der ausführbare strukturelle Code einer Wahrnehmungsinstanz:

$$\Gamma_P = (\mathcal{D}, \mathcal{R}, \mathcal{O}, \mathcal{S}, \mathcal{P}, \mathcal{A}, \mathcal{B}, \mathcal{V}, \mathcal{E}, \mathcal{Q}).$$

$\mathcal{D}$ enthält Observablen und zu unterscheidende Differenzen. $\mathcal{R}$ enthält Relationstypen. $\mathcal{O}$ enthält Ophanprogramme. $\mathcal{S}$ enthält die Skalenleiter. $\mathcal{P}$ enthält Phase-, Rotor- und Carrierprofile. $\mathcal{A}$ enthält Attraktorprogramme. $\mathcal{B}$ enthält Boundary- und Closureverträge. $\mathcal{V}$ enthält View- und Rendergrammatik. $\mathcal{E}$ enthält Evolutionsoperatoren. $\mathcal{Q}$ enthält Quantisierung, Toleranzen und Budgets.

5.2 Gene

Ein Gen ist ein typisierter Genome-Baustein

$$g = (id, type, domain, codomain, params, dependencies, witness, version).$$

Die Referenzgrammatik führt mindestens folgende Genklassen:

- **ResoniteGene**: deterministische Observable auf kanonischen Eingaben;
- **RelationGene**: Typ und Richtung einer strukturellen Relation;
- **OphanGene**: Vorlage eines zyklischen Projektionsorgans;
- **ScaleGene**: Auflösungsstufe, Lift- und Quotientenabbildungen;
- **PhaseGene**: Phase, Rotor, Blatt, Carrier und Synchronisationsprofil;
- **AttractorGene**: Zielklasse, Potential und Stopprädikat;
- **BoundaryGene**: Reflexion, Closure, Seam- und Gluingbedingungen;
- **RenderGene**: HumanView-Mapping, LOD und Styles;
- **EvolutionGene**: zulässige Mutation, Rekombination und Sedimentation.

5.3 Normalform

Die kanonische Normalform lautet

$$NF(\Gamma_P) = Reduce(Canon(Order(Resolve(\Gamma_P)))).$$

Resolve bindet Referenzen und Abhängigkeiten. *Order* erzeugt eine deterministische Reihenfolge. *Canon* normalisiert Repräsentationen. *Reduce* quotiert redundante Gene innerhalb deklarerter Äquivalenzklassen.

Proposition 5.1 (Idempotente Genome-Normalform). *Ist Canon idempotent und Reduce ein idempotenter Quotientenselektor, dann gilt*

$$NF(NF(\Gamma_P)) = NF(\Gamma_P).$$

Genome-Compiler und Perceptual Program

6.1 Compile-Vertrag

Der Compiler erzeugt aus Genome, DomainAdapter und RunDescriptor ein ausführbares Perceptual Program:

$$\mathcal{P}_\Gamma = \text{Compile}(\Gamma_P, DA, RD).$$

Das Ergebnis enthält OphanTemplates, Schedulerregeln, Observablen, Skalenoperatoren, Attraktorpotentiale, Boundary-Operatoren, Viewregeln und Conformance-Checks.

6.2 Kompilationsstufen

Die Referenzpipeline ist

$$\Gamma_P \rightarrow \text{Parse} \rightarrow \text{Resolve} \rightarrow \text{Type} \rightarrow \text{Canon} \rightarrow \text{Link} \rightarrow \text{Plan} \rightarrow \text{Seal} \rightarrow \mathcal{P}_\Gamma.$$

Parse erzeugt den Genome-AST. **Resolve** bindet Referenzen. **Type** prüft Domain/Codomain. **Canon** erzeugt stabile IDs. **Link** verbindet Gene mit Operatorregistry und DomainAdapter. **Plan** erzeugt Scheduler- und Ressourcenpläne. **Seal** erzeugt Digest, Lineage und Replaymanifest.

6.3 Kompilierte Frage

Sensorik wird als kompilierte Frage verstanden. Ein OphanTemplate beschreibt eine Familie von Experimenten, die eine bestimmte Unterscheidung im aktuellen Zustandsraum maximiert. Der Compiler kann dieselbe semantische Frage auf unterschiedliche physische oder virtuelle Träger abbilden, solange deren Adapter dieselbe kanonische Grenzsemantik erfüllen.

6.4 Genome-Paket

Ein distributables Genome-Paket besteht aus

$$\text{Pack} = (\text{Manifest}, \text{Genome}, \text{Types}, \text{OperatorRefs}, \text{Adapters}, \text{Tests}, \text{Digests}, \text{Lineage}).$$

Damit wird eine Wahrnehmungsmodalität als versionierbares, prüfbares Paket adressierbar.

Attraktorprogrammierung

7.1 Strukturelles Potential

Ein Attraktor wird durch das Genome als strukturelle Klasse definiert. Für Observablen f_i und Zielsignaturen g_i verwenden wir

$$V_{\Gamma}(x) = \sum_{i=1}^n w_i d_i(f_i(x), g_i) + \lambda_C C(x) + \lambda_R R(x) + \lambda_S S(x).$$

C erfasst Constraintverletzungen, R Residuenkosten und S Skalen-/Seamkosten. Die Attraktorklasse ist

$$A_{\Gamma} = \arg \min_{x \in \mathcal{X}_{adm}} V_{\Gamma}(x).$$

7.2 Attraktorepoche

Die Laufzeit bindet genau einen primären Attraktorvertrag pro Epoche:

$$A_e = (class, objective, region, stop, horizon, budget, tolerance, scalePolicy).$$

Ein Wechsel erzeugt $e \mapsto e + 1$ und lässt Evidenz, Digests und Weltbindung in ihrer ursprünglichen Lineage bestehen.

7.3 Explorationsattraktor

Ein Genome kann einen explorativen Attraktor tragen, dessen Potential den erwarteten Diskriminationsgewinn, die Erweiterung der geschlossenen Reflexionsregion und die Reduktion relevanter Residuen bewertet:

$$V_{explore}(x) = -\alpha IG(x) - \beta \Delta\mu(\Omega^*) + \gamma Cost(x) + \delta Risk(x).$$

Damit besitzt das System auch bei offener Zielklasse eine wohldefinierte gerichtete Exploration.

Genome-Lineage und Sedimentationsidentität

8.1 Lineagegraph

Genome-Versionen bilden einen gerichteten acyclischen Lineagegraphen. Jeder Knoten trägt Digest, Parentset, Conformance-Suite, Evaluationsbundle und Migrationsoperatoren. Ein Nachfolger bewahrt die kanonische Bedeutung jeder übernommenen Genklasse durch explizite Mappingtabellen.

8.2 Semantische Versionierung

Wir verwenden drei Änderungsklassen. **Patch** verändert Parameter oder Implementierungsprofile innerhalb identischer Grenzsemantik. **Minor** ergänzt neue Gene oder Operatorprofile innerhalb derselben Perceptual-OS-Oberfläche. **Major** verändert eine kanonische Systemrolle oder die Grenzsemantik und erzeugt eine neue Produktklasse.

8.3 Sedimentationswitness

Eine Genome-Version wird sedimentiert, wenn ihre Änderungen durch Runs, Benchmarks, Replay, Ablation und Conformance-Witnesses getragen sind. Der Sedimentationsrecord lautet

$$S_n = (\text{parent}, \text{delta}, \text{evidence}, \text{metrics}, \text{tests}, \text{residues}, \text{canon}, \text{digest}).$$

Dieser Record verbindet theoretische Lineage und implementierbare Versionierung.

Part III

Adaptive Ophan Runtime

Ophan als zyklisches Projektionsorgan

9.1 Konstitution

Ein Ophan ist ein kompilierbares zyklisches Projektionsorgan

$$\mathcal{O}_i = (id, \lambda_i, A_i, \mathcal{R}_i, \Pi_i, \theta_i, \Omega_i, \Gamma_i, G_i, W_i, \tau_i, \rho_i).$$

λ_i bindet das Organ an Zentrum und Genome-Lineage. A_i ist Apertur, $\mathcal{R}_i$ Rotorfaser, Π_i Projektion, θ_i Phase, Ω_i Operatorprogramm, Γ_i Seamvertrag, G_i Gate, W_i Witnessvertrag, τ_i Trace und ρ_i Residuenregel.

9.2 Rotorfaser

Die generische Orientierung wird als Faserbündel modelliert:

$$S^1 \hookrightarrow \mathcal{R} \xrightarrow{\pi} \mathcal{A}.$$

Die Basis $\mathcal{A}$ enthält zulässige Orientierungen oder abstrakte Projektionsrichtungen; die S^1 -Faser trägt die intrinsische zyklische Phase. Räumliche Instanzen können $\mathcal{A} = S^2$ und einen Quaternionenlift verwenden.

9.3 Schwinge

Die momentane gerichtete Ausprägung ist

$$\mathcal{W}_i(t) = \text{Expose}(\mathcal{O}_i, \theta_i(t), B_t, C_t).$$

Die Schwinge trägt Organreferenz, Phase, Achse, Sektor, Orientierung, Payload, Evidenz, Witness und Trace. Sie ist der zur aktuellen Weltbindung ausgewählte Schnitt des zyklischen Organs.

9.4 Open Generative Harmonic Perceptual Field

Die Feldsicht des Systems ist

$$\mathfrak{H}_t = \bigoplus_{i \in I_t} w_i \mathcal{W}_i(t) \otimes h_i(t),$$

mit harmonischer Signatur h_i und gewichteter, dependency-quotierter Kopplung. $\mathfrak{H}_t$ ist die momentane Feldrepräsentation derselben Runtime, die substratseitig als GRFPS geführt wird.

Kohärenzgetriebenes Operatorfeld

10.1 Operatorgraph

Die aktive Ophanbank bildet einen zeitabhängigen gerichteten Graphen

$$G_t = (V_t, E_t, w_t, \phi_t, h_t),$$

mit Operatorinstanzen V_t , Kopplungen E_t , Kantengewichten w_t , Phasen ϕ_t und Historienzuständen h_t . Jeder Knoten trägt lokalen Zustand, Confidence, Ressourcen und Genome-Referenz.

10.2 Lokale Kohärenz

Für Knoten i definieren wir eine lokale Kohärenz

$$c_i(t) = \frac{1}{Z_i} \sum_{j \in \mathcal{N}(i)} w_{ij}(t) \kappa(y_i(t), y_j(t), \Delta\phi_{ij}(t)),$$

mit normierter Kompatibilitätsfunktion κ . Der Feldwert

$$C(t) = \sum_i \omega_i c_i(t)$$

steuert Schedulerprioritäten, Ophanreorientierung und Gluing-Reihenfolge.

10.3 Mehrschichtige Stabilität

Stabilität wird als Vektor geführt:

$$S_i = (S_i^{local}, S_i^{seam}, S_i^{scale}, S_i^{history}, S_i^{target}).$$

Dadurch kann ein Knoten lokal stabil und zugleich skalen- oder zielseitig reorientierungsbedürftig sein. Der Scheduler verwendet diese Komponenten explizit.

10.4 Adaptive Operatorinstanzen

Eine Operatorinstanz besitzt

$$O = (\sigma, \mu, \phi, \chi, v, \Gamma),$$

mit Sensorik, Transformationslogik, Feldwirkung, Instanziierungsregel, Adaptionenregel und Gültigkeitsgrenzen. Die Instanziierungsregel erzeugt eine kontextgebundene Ophaninstanz aus einem GenomeTemplate und koppelt ihren Rückkanal an den globalen Messzustand.

Perzeptiver Scheduler

11.1 Prioritätsfunktion

Für einen Kandidaten o wird

$$\begin{aligned} \text{Score}(o) = & \alpha IG(o) + \beta CovGain(o) + \gamma BoundaryGain(o) + \eta TargetGain(o) \\ & - \lambda Cost(o) - \mu Risk(o) - \nu Redundancy(o). \end{aligned}$$

verwendet. IG misst Diskriminationsgewinn und $CovGain$ den Coveragegewinn. $BoundaryGain$ misst die Erweiterung der geschlossenen Region; $TargetGain$ misst den Attraktorfortschritt.

11.2 Dependencyquotient

Korrelation wird durch einen Dependency-Operator Q_D quotiert. Mehrere ähnlich reagierende Ophanim erhöhen die effektive Evidenz nur entsprechend ihres zusätzlichen Rangs. Der Scheduler priorisiert deshalb unabhängige oder komplementäre Richtungen.

11.3 Schedulingzyklus

Die Referenzfolge lautet

$$Sense \rightarrow Score \rightarrow Select \rightarrow Spawn/Reuse \rightarrow Expose \rightarrow Measure \rightarrow Update \rightarrow Retire.$$

Der Zyklus arbeitet innerhalb eines Budgets und schreibt jede Auswahlentscheidung in den Trace.

11.4 Retirement und Rekonfiguration

Ein Ophan wechselt in einen sedimentierten Inaktivzustand, wenn sein marginaler Informationsbeitrag über ein definiertes Fenster unter die Genome-Schwelle fällt oder seine Funktion von einer kanonisch äquivalenten Instanz übernommen wird. Seine Evidenz- und Lineagereferenzen bleiben adressierbar.

Ausführbare Resonanzsemantik

12.1 Resonite-Funktion

Ein Resonite ist eine deterministische Observable

$$r_j : X \rightarrow Y_j$$

auf einem kanonischen Eingaberaum. Für einen lokalen Patch P erzeugt die Resonitebank den Signaturvektor

$$\mathbf{r}(P) = (r_1(P), \dots, r_m(P)).$$

12.2 Genome-Resonanz

Ein GenomeMotif g trägt Referenzsignatur $\mathbf{g}$, Toleranzmatrix Σ_g und Relevanzgewicht w_g . Die Resonanz ist

$$\mathcal{R}_g(P) = \exp \left[-\frac{1}{2} (\mathbf{r}(P) - \mathbf{g})^\top \Sigma_g^{-1} (\mathbf{r}(P) - \mathbf{g}) \right].$$

Diese Größe ist ein operationales Matchingmaß. Sie speist Attraktorpotential, Ophan-Scheduler und Map-Density.

12.3 Mehrkanalige Resonanz

Für spektrale, topologische, temporale und kausale Kanäle wird

$$\mathcal{R}(P, g) = \prod_{c \in \mathcal{C}} \mathcal{R}_c(P, g)^{\omega_c}$$

verwendet. Die Kanalgewichte sind Genome-Parameter und werden in der Lineage kalibriert.

12.4 Resonanzfeld

Das lokale Resonanzfeld lautet

$$\mathfrak{R}_t(x, s) = \sum_{g \in \Gamma_P} w_g \mathcal{R}_g(P_{x,s,t}).$$

Gradienten und Ridge-Strukturen dieses Feldes liefern Kandidaten für Ophanallokation, Skalenwechsel und Attraktorfortsetzung.

Part IV

Graph-Infogenetische Kartographie

Graphfenster und Strukturresoniten

13.1 GraphWindow

Ein GraphWindow ist ein endlicher lokaler Ausschnitt

$$G_i = (V_i, E_i, \ell_i, \partial_i, origin_i),$$

mit typisierten Knoten und Kanten, lokalen Labels, Boundarymarken und Provenienz. Die Dekomposition wird durch den aktiven DomainAdapter und die Genome-Skalenpolitik bestimmt.

13.2 Fingerprint

Der kanonische Fingerprint ist

$$\Phi(G_i) = (Canon(G_i), Orb(G_i), Stab(G_i), Spec(G_i), TracePow(G_i), Props(G_i), Boundary(G_i)).$$

Diese Mehrsignatur trägt lokale Strukturidentität über mehrere komplementäre Invarianten.

13.3 GraphResonite

Ein GraphResonite ist

$$r_i^G = (origin_i, \Phi(G_i), Gates_i, Residues_i).$$

Es ist das kleinste provenienzgebundene Strukturgen der Karte. Es kann unabhängig gespeichert, verglichen, verklebt und über Lineages hinweg referenziert werden.

13.4 Reproduzierbare Dekomposition

Eine Dekompositionsstrategie D_s ist Teil des RunDescriptors. Gleiche kanonische Eingaben und gleiche Strategie erzeugen dieselbe Menge kanonischer Windows und Digests. Damit bleibt die lokale Kartenbildung replayfähig.

GraphInfogenome und kompatibles Gluing

14.1 GraphInfogenome

Die lokale Kartenrepräsentation ist eine provenienzgebundene Sequenz oder Multimenge

$$\Gamma_t^G = \{(origin_i, r_i^G)\}_{i \in I_t}.$$

Sie ist zugleich Speicherformat für lokale Struktur und Input für die Gluing Engine.

14.2 Patch

Ein Patch ist

$$P_i = (U_i, \Phi(G_i), origin_i, Gate_i, scale_i, time_i).$$

Für $U_i \cap U_j \neq \emptyset$ wird eine Kompatibilität

$$Comp(P_i, P_j; U_i \cap U_j) \in [0, 1]$$

berechnet. Ein SeamWitness materialisiert Übergang, Toleranz, Witnesses und Residuen.

14.3 GluingCertificate

Eine Patchfamilie besitzt das Zertifikat

$$GlueCert(\{P_i\}) = (OverlapGraph, CompatibilityMatrix, Seams, Residues, ReplayDigest).$$

Die Gluing Engine erzeugt daraus einen lokalen Komplex

$$\mathcal{M}_t = Glue(\{P_i\}).$$

14.4 Nervenkomplex

Die Patchfamilie erzeugt einen zeitgebundenen Nerv

$$\mathcal{N}_t = Nerve(\{U_i\}).$$

Vertices repräsentieren Patches, Kanten bezeugte Paarüberlappungen, höhere Simplizes gemeinsame HigherWitnesses. Holonomie und Zyklusstruktur werden als eigene Records geführt.

Perzeptive Massivität und Strukturdichte

15.1 Dichtefunktion

Die lokale perzeptive Strukturdichte ist

$$\rho_P(x, s, t) = \frac{\sum_{i \in I(x, s, t)} w_i q_i c_i e_i}{\mu(\mathcal{H}_{x, s, t}) + \epsilon},$$

mit Gewicht w_i , Witnessqualität q_i , Seamkompatibilität c_i und Evidenzsupport e_i .

15.2 Kartenmaterialisierung

ρ_P steuert die Materialisierung des Synthetic Second Layer. Niedrige Dichte erzeugt weit geöffnete, sparsame Geometrie. Mittlere Dichte erzeugt Linien, Ridge-Strukturen und Flächen. Hohe Dichte erzeugt volumetrisch oder graphisch gebundene Regionskörper. Der RenderCompositor verwendet Dichte zusammen mit Faktizität, Unsicherheit und Skala.

15.3 Dichtegradient

Der Gradient

$$\nabla \rho_P(x, s, t)$$

zeigt lokale Richtungen steigender Strukturbestimmtheit. Er ist ein zusätzlicher Term im Tangentenkompass und im Explorationspotential.

15.4 Dichte und Reflexionsgrenze

Die selbstorganisierte Reflexionsregion bevorzugt kompatible, replaystabile und strukturreiche Gebiete. Eine Dichtekomponente kann deshalb in die Boundary-Optimierung eingehen, während Scope und Witnessbedingungen als harte Constraints geführt werden.

Mirror-Faser und holographische Spiegelklasse

16.1 Latente Klasse

Die Maschine führt eine modellrelative Kandidatenfaser

$$\mathfrak{M}_t = \{\phi \in \Phi_{model} : D_i(O_i(\phi), E_{i,t}) \leq \varepsilon_i \ \forall i \in I_t\}.$$

Aus dieser Faser wird ein kanonischer Repräsentant

$$\hat{\Phi}_t = Canon(\mathfrak{M}_t, E_t, L_t).$$

16.2 Kontraktion

Neue Evidenz e_{t+1} erzeugt

$$\mathfrak{M}_{t+1} = \mathfrak{M}_t \cap C(e_{t+1}).$$

Innerhalb eines fixierten Modell-, Scope- und Toleranzvertrags ist diese Folge monoton bezüglich Mengeninklusion.

16.3 Holographische Rekonstruktion

Viele projektive Schnitte tragen partielle Informationen derselben latenten Struktur. Der ReconstructionOperator kombiniert harmonische Deskriptoren, GraphResonites, Seams, Phasenlagen, Scale-Transport und Historie zu einer Mirror-Faser. Die Rekonstruktion ist damit eine constraint-basierte Synthese lokaler Projektionen.

16.4 Mirror-Lineage

Jede Mirror-Version trägt ModelEpoch, EvidenceDigest, GenomeDigest, ScaleContext, HorizonRef und ParentDigest. Dadurch kann die Fortsetzung einer bereits erworbenen Achse von einer echten Reakquisition unterschieden werden.

Part V

Reflexionsgrenze, Dualphase und Multiskaligkeit

Selbstadjungierte Reflexionsgrenze

17.1 Reflexionsoperator

Die lokale Reflexionsstruktur wird durch eine selbstadjungierte Involution geführt:

$$R_t = R_t^*, \quad R_t^2 = I.$$

Daraus entstehen komplementäre Projektoren

$$B_t^+ = \frac{1}{2}(I + R_t), \quad B_t^- = \frac{1}{2}(I - R_t).$$

Sie bilden die dualphasige Zerlegung einer gemeinsamen Grenzstruktur.

17.2 Geschlossene Region

Der aktuelle operative Kern ist eine Region $\Omega_t^* \subseteq \mathcal{M}_t$, die Closure-, Gluing-, Replay-, Pfad- und Reflection-Witnesses erfüllt. Eine idealisierte Auswahl ist

$$\Omega_t^* = \arg \max_{U \subseteq \mathcal{M}_t} [\mu(U) + \lambda_\rho \bar{\rho}_P(U) + \lambda_W W(U)]$$

unter

$$C_{glue}(U) = C_{replay}(U) = C_{path}(U) = C_{reflect}(U) = C_{scope}(U) = 1.$$

Die operative Reflexionsgrenze ist

$$\Sigma_t = \partial\Omega_t^*.$$

17.3 Boundary-Renewal

Neue Patches können die geschlossene Region erweitern. Ein Renewal-Schritt berechnet

$$\Omega_{t+1}^* = \text{Renew}(\Omega_t^*, \Delta M_t, \Gamma_t).$$

Der Schritt erzeugt einen BoundaryWitness, aktualisiert den Horizon und bindet die neue Region an den Replaydigest.

17.4 Selbstadjungierter Center-Frame

Das markierte Zentrum induziert einen laufgebundenen Reflexionsframe R_z . CenterBinding, WorldBinding und Genome definieren gemeinsam, welche lokalen Transformationen als gleiche strukturelle Orientierung gelten.

Dualphase als Grenzkörper

18.1 Bifazialer Zustand

Ein dualphasiger Perzeptionszustand wird als Faserprodukt geführt:

$$M_t = F_t^+ \times_{N_t} F_t^-.$$

F_t^+ und F_t^- sind synchronisierte, typisierte Felder über einer gemeinsamen Referenzstruktur N_t . Die Cross-Phase-Relation trägt Information, die in einem einzelnen Feld verborgen bleiben kann.

18.2 Partielle Korrespondenz

Die Zuordnung

$$\beta_t : A_t^+ \rightarrow A_t^-$$

ist partiell und scopegebunden. Ungepaarte Anteile werden als Residuen, Horizon oder neue Sondenpflicht geführt. Die Seam ist der Witness der lokalen Kompatibilität beider Seiten.

18.3 Cross-Phase-Disparität

Für identifizierte Paare definieren wir

$$\Delta_{+-}(x) = D(I_+(x), \beta_t^{-1}(I_-(x))).$$

Diese Disparität ist ein eigener Deskriptor des Sensoriums. Sie kann Attraktoren, Boundary-Kandidaten und Achsenwechsel markieren.

18.4 Triphasische Prozessordnung

Auf Prozessebene kann eine dreigliedrige Kommutation verwendet werden:

$$Acquire \rightarrow Coherence \rightarrow Commit \rightarrow Acquire.$$

Sie organisiert Aufnahme, Feldbildung und Sedimentation als zyklische Runtimeordnung und ist unabhängig von der Anzahl gleichzeitig geführter Perzeptionsfelder.

Phasenhorizont

19.1 Horizonttupel

Der Phasenhorizont ist

$$\mathcal{H}_t = (H_t^{cur}, H_t^{cf}, H_t^{open}).$$

H^{cur} enthält aktuell zulässige Fortsetzungen, H^{cf} kontrafaktische oder später aktivierbare Fortsetzungen, H^{open} strukturell adressierte Bereiche mit offener Bestimmtheit.

19.2 HorizonChart

Ein HorizonChart trägt

$$HC = (center, scope, scale, phaseRays, projectionCones, reachability, boundary, assumptions, trace).$$

PhaseRays bilden deterministische Trägerlinien vom markierten Zentrum in den lokalen Operationsraum. ProjectionCones bündeln zulässige Sonden- und Fortsetzungsrichtungen.

19.3 Horizontdynamik

Jeder Puls aktualisiert den Horizont über neue Evidenz, Boundary-Renewal und Attraktorepoche:

$$H_{t+1} = UpdateHorizon(H_t, E_{t+1}, \Omega_{t+1}^*, A_e).$$

Die HumanView kann alle drei Teilhorizonte verschieden materialisieren.

19.4 720-Grad-Semantik

Die 720-Grad-Semantik wird als Kombination einer sichtbaren Orientierungsclosure und einer gelifteten Verlaufsklasse geführt. Im räumlichen Profil gilt

$$SU(2) \rightarrow SO(3), \quad \pi(q) = \pi(-q).$$

Der Lift bewahrt die Verlaufsklasse, während der sichtbare Frame die aktuelle Projektion trägt.

Multiskaliges Perzeptionssubstrat

20.1 Skalenposet

Die Skalenstruktur ist ein geordnetes Set

$$(\mathcal{S}, \preceq) = \{s_0, s_1, \dots, s_k\}$$

mit beispielhaften Rollen Mikro, Meso und Makro. Jede Skala besitzt eigenen Patchraum, Horizon, Dichte, Atlas und Attraktorrepräsentation.

20.2 Lift und Quotient

Für $s \preceq s'$ existieren typisierte Abbildungen

$$L_{s \rightarrow s'} : \mathcal{M}^{(s)} \rightarrow \mathcal{M}^{(s')}, \quad Q_{s' \rightarrow s} : \mathcal{M}^{(s')} \rightarrow \mathcal{M}^{(s)}.$$

Ein ScaleRoundtripWitness prüft

$$Q_{s' \rightarrow s}(L_{s \rightarrow s'}(x)) \sim_s x$$

innerhalb des deklarierten Toleranzprofils.

20.3 Skalenfaser

Der Gesamtoperationsraum ist

$$\mathfrak{M} = \bigsqcup_{s \in \mathcal{S}} \mathcal{M}^{(s)}$$

mit Scale-Seams zwischen kompatiblen lokalen Charts. Horizontnavigation findet damit sowohl innerhalb einer Skala als auch zwischen Skalen statt.

20.4 Multiskaliger Attraktor

Ein Attraktor ist

$$A_e = \{A_e^{(s)}\}_{s \in \mathcal{S}}$$

mit Cross-Scale-Compatibility $\kappa_{s,s'}$. Der Navigator kann deshalb eine lokale Fortsetzung oder einen Skalenwechsel vorschlagen.

Gyrosphärische Coverage und Scale-Pose

21.1 Gyrosphärenzellen

Die Gyrosphäre besteht aus orientierten Zellen

$$g_j = (id_j, q_j, u_j, A_j, O_j, w_j, c_j, s_j, r_j, \tau_j).$$

Zusätzlich zur gelifteten Orientierung q_j und sichtbaren Richtung u_j trägt jede Zelle eine aktive Skala s_j .

21.2 Coverageoperator

Für räumliche Profile ist

$$Cov_t : S^2 \times \mathcal{S} \rightarrow [0, 1]$$

eine gewichtete Richtungs- und Skalenabdeckung. Allgemein wird S^2 durch den Domänenorientierungsraum ersetzt.

21.3 Frameoperator

Für Richtungsvektoren u_j und Gewichte w_j definieren wir

$$F_t = \sum_j w_j u_j u_j^\top.$$

Eigenwertverteilung und condition number des Frameoperators messen globale Anschlussfähigkeit aus lokal anisotropen Zellen.

21.4 Reorientierungsenergie

Eine Referenzenergie lautet

$$\mathcal{E}_t = \lambda_D D_{data} + \lambda_S D_{seam} + \lambda_C D_{coverage} + \lambda_A V_{A_e} + \lambda_\rho D_{density} + \lambda_\sigma D_{scale} + \lambda_U U_t + \lambda_B Cost_t.$$

Ein Relaxationsoperator aktualisiert Orientierung und Scale-Pose unter Genome- und Boundary-Constraints.

Part VI

Attraktornavigation und Synthetic Second Layer

Tangentenkompass und Skala

22.1 Lokaler Tangentialraum

Im aktuellen Chart $x_t \in \mathcal{M}^{(s_t)}$ besitzt der Navigator einen lokalen Fortsetzungsraum $T_{x_t} \mathcal{M}^{(s_t)}$. Das Attraktorpotential liefert die horizontale Komponente

$$v_t^{\parallel} = -\Pi_{adm} \nabla_x V_{A_e}(x_t, s_t).$$

22.2 Skalenrichtung

Die vertikale Komponente ist eine diskrete oder kontinuierliche Scale-Empfehlung

$$v_t^{\perp} = \arg \min_{s' \in Adj(s_t)} \mathbb{E}[V_{A_e}(x', s') + Cost_{scale}(s_t, s')].$$

Der vollständige Kompass ist

$$v_t = (v_t^{\parallel}, v_t^{\perp}).$$

22.3 Richtungsfächer

Mehrere annähernd gleichwertige Fortsetzungen erzeugen einen Fächer

$$\mathcal{V}_t = \{(v_t^{(k)}, p_k, scale_k)\}_{k=1}^m.$$

Marge, Witnessrang und Horizonstatus bestimmen die sichtbare Schärfe des Fächers.

22.4 Axis Lock

Eine Achsenklasse wird gelockt, wenn Separationsmarge, Witnesssupport, Persistenz und Attraktorepochenbindung die Genome-Schwelle erfüllen. Danach wird die Achse durch Paralleltransport fortgesetzt und mit neuer Evidenz korrigiert.

Aktive Navigation

23.1 Minimaler Benutzeroperator

Im Minimalmodus besitzt der Benutzer einen Vortriebsparameter

$$u_t \in [0, 1].$$

Der AdvanceOperator berechnet

$$x_{t+1} = \textit{Advance}(x_t, u_t, v_t, \mathcal{H}_t, \Omega_t^*).$$

Der konkrete Effekt von Advance wird vom DomainAdapter bestimmt: physische Pose, Sondenwahl, Datenschnitt, Parameteränderung, Wartefenster oder rein virtuelle Exploration.

23.2 ALIGN–LOCK–ADVANCE

Die Experience-Normalform lautet

$$\textit{ALIGN} \rightarrow \textit{LOCK} \rightarrow \textit{ADVANCE} \rightarrow \textit{REOBSERVE} \rightarrow \textit{CORRECT}.$$

ALIGN reduziert Framefehler. LOCK bindet eine fortsetzbare Achsenklasse. ADVANCE führt einen zulässigen Schritt aus. REOBSERVE aktualisiert Evidenz und Karte. CORRECT aktualisiert Achse, Horizon und Attraktorpotential.

23.3 WorldlineWindow

Ein Weltlinienfenster bindet einen diskreten Ereignisanker mit mehreren Registern:

$$W = (\textit{anchor}, \textit{observed}, \textit{reconstructed}, \textit{counterfactual}, \\ \textit{projected}, \textit{orientations}, \textit{channels}, \textit{status}).$$

Es verbindet Ereigniszeit, Trace, Modellrekonstruktion und prospektive Fortsetzung in einem einzigen getypten Objekt.

23.4 Ratchet

Für eine endliche Kandidatenmenge S_0 erzeugt der NavigationRatchet entweder eine echte Kontraktion $S_{n+1} \subsetneq S_n$ oder einen typisierten Endstatus. Damit entsteht eine endliche, replaybare lokale Closure.

Synthetic Second Layer

24.1 Framekette

Die Darstellung verwendet drei explizite Frames:

$$F_W \xrightarrow{T_{W \rightarrow M}} F_M \xrightarrow{T_{M \rightarrow H}} F_H.$$

F_W ist Weltframe, F_M Maschinenframe, F_H Human-/Headsetframe. Jede sichtbare Struktur trägt ihre Frame-Lineage.

24.2 Grundobjekte

Der Compositor kennt mindestens folgende sichtbare Objekte:

- Center Marker und Frameachsen;
- Phasenhorizont und Horizonsegmente;
- Attraktormarker und Potentialridge;
- Tangentenpfeil, Richtungsfächer und Scale-Arrow;
- Seam, Boundary und ReflectionShell;
- Strukturdichtefelder, Graphkanten und lokale Flächen;
- Residuen, offene Patches und Reacquisitionmarker;
- Tracepfade und Replayghosts.

24.3 Rendergrammatik

Die Rendersemantik ist eine Funktion

$$Render : (MachineObject, Facticity, Uncertainty, Scale, Density, Age) \rightarrow VisualObject.$$

GenomeRenderGenes definieren LOD, Opazität, Geometrieklasse, Animation, Textur, Labeling und Interaktionskanäle.

24.4 Perzeptive Massivität

Die Strukturdichte ρ_P wird in eine visuelle Materialität überführt. Offene Regionen besitzen geringe geometrische Bindung. Mit wachsender Dichte entstehen Kanten, Flächen, Volumina und navigierbare Strukturen. Dadurch wächst die sichtbare Landschaft proportional zur rekonstruierten Struktur statt zur bloßen Datenmenge.

Phasenhorizont als Navigationspanorama

25.1 Panorama

Der Phasenhorizont wird als 360-Grad- beziehungsweise domänenvollständiges Panorama des aktuellen Fortsetzungsraums gerendert. Current-, Counterfactual- und Open-Komponenten besitzen getrennte ViewChannels und gemeinsame Provenance.

25.2 Horizontmarker

Jeder Marker ist

$$h = (direction, scale, status, reachability, expectedGain, uncertainty, sourceRefs).$$

Ein Marker kann Punkt, Fächer, Ring, Korridor oder Flächenkörper sein. Seine Geometrie repräsentiert die Form der Fortsetzungsmenge.

25.3 Horizon Expansion

Neue Ophanim können gezielt in Regionen mit geringer Coverage oder hoher Attraktorrelevanz instanziiert werden. Die resultierenden Patches erweitern Karte und Boundary. Horizon Expansion ist deshalb ein unmittelbarer Effekt der Perzeptionsruntime.

25.4 Reflexionsgrenze als sichtbare Hülle

Die Boundary kann als Hülle, Seam, Glasfläche, Halo, Kontur oder vollständig abstrakte UI-Schicht projiziert werden. Ihre sichtbare Bedeutung ist der Rand des aktuell geschlossenen, replaybaren Maschinenraums.

Part VII

Langsame Evolution und Selbstkompilation

Peristaltischer Sedimentationszyklus

26.1 Slow Loop

Der langsame Zyklus verarbeitet versiegelte Laufserien statt einzelne Sensorevents. Sein Zustand ist Λ_n . Eine Iteration lautet

$$\Lambda_{n+1} = \text{Sediment}(\Lambda_n, \text{Runs}_n, \text{Metrics}_n, \text{Residues}_n, \text{Ablations}_n).$$

26.2 Phasen

Die Referenzfolge ist

$$\text{Ingest} \rightarrow \text{Observe} \rightarrow \text{Salvage} \rightarrow \text{Normalize} \rightarrow \text{Stratify} \rightarrow \text{Harden} \rightarrow \text{Reconstruct} \rightarrow \text{Seal}.$$

Ingest bindet Daten und Lineage. Observe zerlegt Runs in Operator-, Genome- und Metrikereignisse. Salvage identifiziert stabile Strukturfunktionen. Normalize erzeugt kanonische Typen. Stratify ordnet Änderungsrollen. Harden erzeugt Tests und Witnesses. Reconstruct kompiliert die neue Genome-/Operatorversion. Seal erzeugt Digest und Migration.

26.3 Runtime zu Lineage

Runtimeartefakte werden erst über einen SedimentationCandidate in die Slow Loop promoviert:

$$SC = (\text{pattern}, \text{support}, \text{domains}, \text{metrics}, \text{ablation}, \text{replay}, \text{residues}).$$

Damit bleiben spontane Laufzustände und strukturelle Evolution getrennte Zeitskalen.

26.4 Lineage-Fixpunkt

Ein Lineage-Fixpunkt liegt vor, wenn neue Daten innerhalb des aktuellen Conformance- und Genomevertrags ausschließlich Parameter- oder Profilupdates erzeugen. Die kanonische Systemoberfläche bleibt dabei stabil.

Genome-Morphogenese

27.1 Mutation

Eine Mutation verändert ein Gen innerhalb seines Typs:

$$\mu_g : g \mapsto g'.$$

Beispiele sind Toleranzupdate, neue Parameter, neue Resonanzkoeffizienten oder verfeinerte Budgetprofile.

27.2 Rekombination

Rekombination verbindet kompatible Genfamilien:

$$\chi : (G_A, G_B, SeamWitness) \mapsto G_C.$$

Der resultierende Genblock trägt Parentrefs und einen CompatibilityWitness.

27.3 Spezialisierung

Ein allgemeines OphanGene kann über einen DomainAdapter in eine spezialisierte Vorlage überführt werden:

$$Spec_D(g_O) = g_O^D.$$

Die Spezialisierung bewahrt die kanonische Grenzsemantik und ergänzt domänenspezifische Parameter.

27.4 Morphogenesecontroller

Der Controller bewertet CandidateGenes über Utility, Transfer, Replay, Robustheit, Strukturgewinn und Komplexitätskosten. Gewinner werden in Shadow-Versionen kompiliert und gegen Referenzruns geprüft. Ein erfolgreicher Kandidat wird als neue Genome-Lineage sedimentiert.

Meta-Perzeption und Selbstmodell

28.1 Runtime-Selbstmodell

Das System führt ein operationales Selbstmodell

$$SM_t = (GenomeState, OphanState, Coverage, Boundary, Atlas, Metrics, Drift, Resources).$$

Es beschreibt die eigene aktuelle Wahrnehmungsfähigkeit innerhalb des Run- und Genomevertrags.

28.2 Capability Map

Die Capability Map ordnet Perzeptionsfunktionen ihren beobachteten Leistungsbereichen zu:

$$Cap : TaskClass \times Domain \times Scale \rightarrow (accuracy, latency, coverage, uncertainty).$$

Sie steuert Scheduler, View und spätere Capability-Envelopes.

28.3 Meta-Attraktor

Ein Meta-Attraktor kann auf die Verbesserung einer Wahrnehmungsfähigkeit gerichtet sein, etwa auf Coverage, Identifizierbarkeit oder Boundary-Wachstum. Solche Attraktoren operieren in der Slow Loop und erzeugen GenomeCandidate-Versionen.

28.4 Selbstmodell und HumanView

Der Compositor kann ausgewählte Capability- und Driftinformationen darstellen. Dadurch wird sichtbar, welche Teile der aktuellen Karte stark getragen, frisch erworben, im Scale-Transport oder in Reacquisition sind.

Part VIII

Maschinenvertrag und Implementierung

Runtime-Module

29.1 Modulgraph

Die Referenzimplementierung besitzt zwölf Runtime-Module und eine Lineage Engine:

ID	Modul	Kernfunktion
K00	Constitution Kernel	Run, Typen, Canon, Digest, Statusalgebra
K01	World Binder	Center, Frames, Timebase, Apertur, Kalibrierung
K02	Genome Compiler	Parse, Resolve, Type, Link, Plan, Seal
K03	Ophan Runtime	Spawn, Activate, Expose, Measure, Retire
K04	Coherence Scheduler	Operatorgraph, Scores, Dependencies, Budgets
K05	Infogenetic Mapper	GraphWindows, Resonites, GraphInfogenome, Gluing
K06	Mirror Engine	Candidate Fiber, Evidence Contract, Axis Candidates
K07	Reflection Engine	R , $B^\pm$, Ω^* , Boundary Renewal
K08	Gyro/Scale Engine	Orientation, Coverage, Scale-Pose, Holonomy
K09	Navigator	Attractor, Potential, Horizon, Compass, Ratchet
K10	View Compositor	Second Layer, LOD, density, facticity, interactions
K11	Ledger/Replay	Trace, checkpoints, receipts, replay classes
K12	Effect Port	Plan, authorization, attempt, receipt, reconciliation
L00	Sedimentation Engine	Run bundles, genome evolution, schema lineage

29.2 Modulabhängigkeiten

K00 und K01 bilden den Maschinenboden. K02 kompiliert K03–K10. K03–K09 erzeugen den internen Perzeptionsraum. K10 projiziert. K11 persistiert alle normativen Übergänge. K12 koppelt optional externe Effekte. L00 arbeitet auf versiegelten Bundles aus K11.

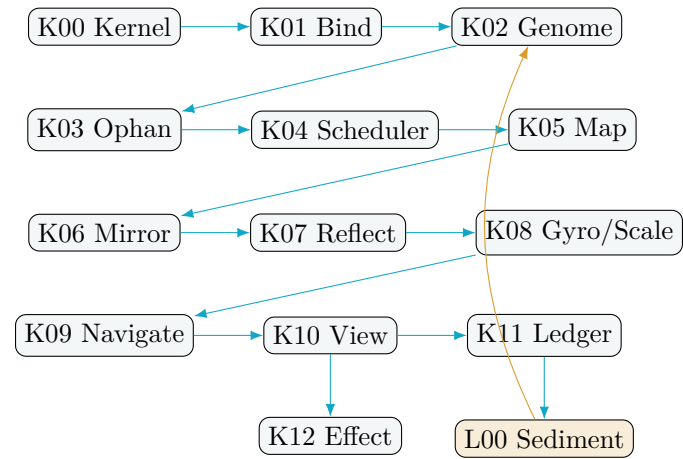


Figure 29.1: Referenzmodulgraph des Perzeptionsbetriebssystems.

Kernrecords

30.1 Recordfamilien

Die Referenzimplementierung führt kanonische Records. Isomorphe interne Repräsentationen sind zulässig, solange alle Pflichtfelder und Grenzsemantiken erhalten bleiben.

Record	Mindestinhalt
RunDescriptor	run_id, version, identity_digest, domain, models, tolerances, budgets, policies, seeds, clocks, adapters, conformance
KernelState	run_ref, update_index, state_digest, status, active_epochs, ledger_ref
CenterBinding	center_id, lineage, validity, trace_ref
WorldBinding	anchor, world_frame, machine_frame, human_frame, aperture, calibration, uncertainty
PerceptualGenomeRecord	genome_id, version, genes, scale_lattice, attractor_programs, boundary_programs, render_grammar, digest
GenomeCompileManifest	genome_ref, resolved_refs, type_report, linked_operators, scheduler_plan, digest
OphanSpec	type, aperture, rotor_profile, projection, operator_graph, witness_contract, budgets
OphanState	id, spec_digest, center_binding, rotor_state, phase_lift, evidence, residues, trace
ExposureState	ophan_ref, phase, axis, sector, sheet, orientation, payload, witnesses
HarmonicDescriptor	profiles, windows, coefficients, spectra, path_signatures, uncertainty, provenance
CoherenceNode	operator_ref, local_state, phase, coherence, stability_vector, resource_state
CoherenceEdge	source, target, weight, relation_type, dependency, seam_ref
GraphWindowRecord	origin, scale, window_id, canonical_graph, boundary, digest
GraphResoniteRecord	origin, canon, orbit, stabilizer, spectrum, trace_powers, properties, boundary, gates
GraphInfogenomeRecord	members, provenance_graph, normal_form, digest
SeamWitness	charts, overlap, transition, compatibility, tolerance, witnesses, trace

NerveState	vertices, simplices, higher_witnesses, seams, holonomy_refs, digest
PerceptualDensityField	chart, scale, density_samples, estimator, uncertainty, digest
MirrorState	candidate_class, model_epoch, evidence_constraints, axis_locks, harmonic_ref, nerve_ref, horizon_ref, residuals, digest
ReflectionState	reflection_operator, plus_projector, minus_projector, region_ref, boundary_ref, witness_refs
BoundaryRegion	region_id, member_patches, closure_witnesses, measure, density, lineage, digest
HorizonState	current, counterfactual, open, assumptions, reachability, validity
AtlasState	charts, transitions, seams, holonomies, worldline, compression, lineage, digest
ScaleState	scale_id, parent, children, lift_ops, quotient_ops, roundtrip_witnesses
GyroState	lifted_orientation, visible_orientation, cells, coverage_ref, scale_pose, attractor_epoch, holonomy, closure
CoverageReport	domain, scale, covered, holes, redundancy, dependency_rank, uncertainty
AttractorSpec	target_class, objective, admissible_region, stop, horizon, budget, tolerance, scale_policy
AxisLock	axis_class, margin, witnesses, persistence, attractor_epoch, transport_ref, confidence, status
NavigationProposal	target_epoch, axis_ref, direction_or_fan, scale_action, alternatives, expected_progress, uncertainty
HumanView	frame, clock, objects, facticity_styles, horizon_styles, scale_styles, provenance_map, trace_ref
TraceSegment	parent_digest, input_digest, operator, pre_digest, post_digest, witnesses, residuals, clocks, determinism
ReplayManifest	run_descriptor_ref, input_sequence, checkpoints, expected_digests, tolerance_profile, result
SedimentationCandidate	pattern, support, domains, metrics, ablation, replay, residues
GenomeLineageRecord	genome_id, parents, delta, migration, conformance, evidence, digest

Operatorregistry

31.1 Operatorfamilien

Die Registry gruppiert Operatoren nach Maschinenrolle.

Familie	Referenzoperatoren
Constitution	bindRun, bindCenter, bindWorld, canon, digest, type, anchor, seal
Genome	parseGenome, resolveGenes, typeGenome, canonicalizeGenome, linkGenome, planGenome, sealGenome, migrateGenome
Observation	ingest, calibrate, normalizeEvidence, classifyFacticity, compileCausalBody, updateBoundaryInput
Ophan	scheduleOphan, compileOphan, spawnOphan, activateOphan, expose, project, measure, retireOphan
Coherence	buildOperatorGraph, updateCoherence, quotientDependencies, scoreCandidate, allocateBudget, reconfigureGraph
Infogenetic Map	decomposeWindow, canonGraph, fingerprintGraph, emitGraphResonite, updateGraphInfogenome, compareOverlap, witnessSeam, gluePatches, buildNerve
Mirror	updateMirror, contractEvidence, compareCandidates, computeMarginalGain, estimateIdentifiability
Reflection	buildReflection, projectPlus, projectMinus, optimizeRegion, renewBoundary, certifyClosure
Scale	bindScale, liftScale, quotientScale, compareScaleRoundtrip, migrateScale, estimateScaleGain
Gyrosphere	orient, relax, splitCell, mergeCell, measureCoverage, closeLift, measureHolonomy
Navigation	bindAttractor, switchAttractor, compilePotential, buildHorizon, discoverAxis, lockAxis, trackAxis, reacquireAxis, buildCompass, advance, ratchet
View	buildViewObject, renderDensity, renderHorizon, renderCompass, renderScale, registerFrames, composeView
Ledger	appendTrace, checkpoint, replay, compareReplay, emitDigest, bindResidue
Effect	planProbe, authorize, issueToken, attempt, ingestReceipt, reconcile

Sedimentation	collectRuns, extractPattern, evaluateCandidate, mutateGenome, recombineGenes, shadowCompile, compareLineage, sedimentVersion
---------------	--

31.2 Operatorvertrag

Jeder Operator besitzt

$$o = (D_o, C_o, Pre_o, Post_o, Det_o, \beta_o, E_o, R_o, W_o, T_o).$$

Die Registry ist selbst content-adressiert. GenomeGene-Referenzen binden konkrete Operatorversionen über Digests.

Conformance und Fixpunktwitness

32.1 Perceptual-OS Conformance

Eine Implementierung besteht die v3-Konformität, wenn sie mindestens folgende Witnessfamilien erzeugt:

1. SchemaWitness: kanonische Typen, Roundtrips und Digests;
2. GenomeWitness: Normalform, deterministischer Compile-Plan und Lineage;
3. OphanWitness: Spawn, Exposition, Trace und Budgetbindung;
4. CoherenceWitness: dependency-quotierter Operatorgraph und Schedulerentscheidungen;
5. GraphWitness: reproduzierbare Windows, Multi-Signatur und Provenienz;
6. GlueWitness: kompatible Patches, Seams, Nerve und Replaydigest;
7. BoundaryWitness: selbstadjungierte Reflexion, geschlossene Region und Renewal;
8. ScaleWitness: Lift/Quotient-Roundtrip und Cross-Scale-Compatibility;
9. NavigationWitness: Attraktorepoche, Axis Lock, Tangentenkompass und Ratchet;
10. ViewWitness: Framebindung, Provenance Map, Dichte- und Horizonprojektion;
11. ReplayWitness: gleiche kausale und kanonische Laufklasse;
12. SedimentationWitness: Runbundle zu Genome-Lineage mit Migrationsrecord.

32.2 Perzeptiver Fixpunkt

Ein Lauf erreicht lokalen Fixpunkt, wenn

$$\text{Can}(\Sigma_{t+1}) = \text{Can}(\Sigma_t)$$

unter fixiertem Attraktor-, Genome-, Scope- und Toleranzvertrag, der maximale budgetierte Strukturgewinn des Schedulers unterhalb der gesetzten Schwelle liegt, die aktive Boundary-Region zertifiziert ist, der Ratchet terminiert und Replay dieselbe Grenzklasse reproduziert.

32.3 Perceptual-OS Fixpoint

Der Systemfixpunkt dieser Monografie liegt vor, wenn alle zwölf Runtimeflächen und die Lineage Engine über die gemeinsame Kerneloberfläche geschlossen implementierbar sind. Das resultierende System ist ein vollständiger Maschinenentwurf und bildet die Ausgangsbasis für den Referenzkern.

Versiegelte Referenzwelten

33.1 Synthetic World Contract

Eine Referenzwelt ist

$$W^* = (X, F, O^*, A^*, S^*, B^*, GT, Noise, Clock).$$

GT enthält versiegelte Ground Truth, O^* die zulässigen Beobachtungsoperatoren, A^* bekannte Attraktoren, S^* Scale-Struktur und B^* Boundary-Topologie.

33.2 Phantomfamilien

Die Referenzsuite umfasst mindestens folgende Familien: zyklische Rotorwelt, kospektrale Graphdoppel, partiell überlappende Patchwelt, driftende Achse, Skalenaliasing, dualphasige Seamwelt, Horizon-Lücken, bewegliche Attraktoren, temporale Weltlinienfenster, korrelierte Sondenbanken, false-glue Kandidaten und reversible Scale-Lifts.

33.3 Blindablauf

Ein Lauf erhält Weltadapter, Genome, Attraktorvertrag und Budgets. Ground Truth bleibt bis zum Seal getrennt. Nach Seal werden Mirror, Karte, Boundary, Attraktorankunft, Horizon, Axis, Scale, View und Replay gegen die Ground Truth ausgewertet.

33.4 Metriken

Die Kernmetrikfamilie umfasst Identifizierbarkeit, Calibration, Witnessrank, PatchPrecision, GluePrecision, BoundaryCoverage, ScaleRoundtripError, AxisPersistence, CompassProgress, HorizonRecall, DensityCalibration, ReplayIdentity und SedimentationTransfer.

Vertikale Realisierung

34.1 Phasen

Die Realisierung wird in acht Phasen mit 32 kumulativen Stufen gegliedert.

Stufe	Name	Sediment
R00	Identity Lock	v3-Grenzoberfläche, 12+1 Zustandsflächen, Digests
R01	Schema Lock	kanonische Schemas, Serialisierung, Typen
R02	Kernel Skeleton	Statusalgebra, Registry, Trace, Replay
R03	Empty Run	leerer vollständiger Run mit Seal
R04	Genome AST	Parser, Gene, Referenzen, Normalform
R05	Genome Compiler	Linker, Planer, CompileManifest
R06	Ophan Template	zyklisches Einzelorgan im Synthetic Adapter
R07	Wing Runtime	gerichtete Exposition, Phase, Trace
R08	Multi-Ophan Bank	finite adaptive Bank, Budgets, Reuse
R09	Coherence Graph	Knoten, Kanten, lokale Kohärenz, Historie
R10	Scheduler	IG/Coverage/Boundary/Target-Scoring
R11	Dependency Quotient	effektiver Rang und Redundanzmodell
R12	GraphWindow Scanner	deterministische lokale Dekomposition
R13	GraphResonites	Canon/Orbit/Stab/Spec/TracePow/Props
R14	GraphInfogenome	Provenienz, Normalform, Digests
R15	Gluing Engine	Overlap, SeamWitness, GlueCert
R16	Nerve Complex	HigherWitnesses, Holonomie, lokale Topologie
R17	Mirror Engine	Kandidatenfaser, Kontraktion, MarginalGain
R18	Reflection Shell	R , $B^\pm$, Ω^* , Renewal
R19	Phase Horizon	Current/Counterfactual/Open Charts
R20	Scale Lattice	Mikro/Meso/Makro, Lift/Quotient
R21	Gyrosphere	Quaternionenlift, Coverage, Scale-Pose
R22	Axis Engine	Discovery, Lock, Transport, Reacquisition
R23	Tangent Compass	horizontaler und vertikaler Navigationsvektor
R24	Attractor Runtime	Genomepotential, Epoch, Ratchet, Arrival
R25	Density Renderer	Strukturdichte, LOD, Map-Materialisierung
R26	Desktop Second Layer	Framekette, Horizon, Compass, Scale UI

R27	Blind Integration Witness	End-to-End in versiegelten Welten
R28	Sedimentation Engine	Runbundles, CandidateGenes, Genome-Lineage
R29	Real-time Carrier Interface	kontinuierlicher Ingest, Pose, Clock, Latenz
R30	XR Prototype	reales Headset, Tracking, Kalibrierung, Occlusion
R31	Capability Release	vermessenes Capability Envelope und Reference Release

34.2 Erster Implementierungszug

Der unmittelbar ausführbare Zug umfasst R00 bis R05. Danach existiert ein kanonischer Maschinenboden mit Perceptual Genome, Compiler, Registry, Trace und Replay. R06 bis R11 materialisieren das adaptive Ophanfeld. R12 bis R19 erzeugen die selbstorganisierte Karte und Reflexionsgrenze. R20 bis R27 materialisieren die Navigation. R28 bis R31 schließen Lineage, Echtzeit und XR.

34.3 Releasekriterium

Jede Stufe erzeugt einen ExitDigest, ein ConformanceBundle und einen adressierten Restbestand. Der Nachfolger verwendet ausschließlich versiegelte Vorgängerartefakte als normative Inputs.

Part IX

Kanonische Abschlussgestalt

Gesamtnormalform

35.1 Run-Normalform

Die vollständige Runtime kann in folgender Kette gelesen werden:

$$\begin{aligned}
 D_{raw} &\xrightarrow{Bind/Canon} (B, C) \xrightarrow{Load/Compile} \Gamma \xrightarrow{Spawn/Schedule} O, \\
 O &\xrightarrow{Expose/Measure} E \xrightarrow{Graphify/Fingerprint} \Gamma^G \xrightarrow{Glue} M, \\
 M &\xrightarrow{Mirror/Reflect} (P, R) \xrightarrow{Scale/Gyro} Y \xrightarrow{Attract/Navigate} N, \\
 N &\xrightarrow{Render} V \xrightarrow{Trace/Replay} L.
 \end{aligned}$$

35.2 Slow-Normalform

Über versiegelten Runs liegt

$$\{L^{(1)}, \dots, L^{(m)}\} \xrightarrow{Collect/Evaluate} SC \xrightarrow{Mutate/Recombine} \Gamma' \xrightarrow{ShadowCompile} \mathcal{P}_{\Gamma'} \xrightarrow{Conformance} \Lambda_{n+1}.$$

35.3 Substrat- und Feldsicht

Die Substratsicht beschreibt Records, Operatoren, Graphen, Boundary und Runtime. Die Feldsicht beschreibt

$$\mathfrak{H}_t = Field(\Sigma_t)$$

als Open Generative Harmonic Perceptual Field. Beide Ansichten sind kanonische Projektionen derselben Maschinenidentität.

Perzeptionsbetriebssystem als vollständiger Maschinenentwurf

36.1 Betriebszyklus

Der vollständige innere Zyklus lautet

$$\begin{aligned} & BIND \rightarrow LOAD \rightarrow COMPILE \rightarrow SPAWN \rightarrow SENSE, \\ & \rightarrow GRAPHIFY \rightarrow GLUE \rightarrow REFLECT \rightarrow ORIENT \rightarrow ATTRACT, \\ & \rightarrow NAVIGATE \rightarrow VIEW \rightarrow TRACE \rightarrow REPLAY. \end{aligned}$$

Bei Lineage-Ereignissen folgt

$$COLLECT \rightarrow EVALUATE \rightarrow SEDIMENT \rightarrow VERSION \rightarrow RECOMPILE.$$

36.2 Perzeptive Selbstorganisation

Selbstorganisation entsteht aus vier gekoppelten lokalen Mechanismen: adaptive Ophanallokation, kohärenzgetriebene Graphdynamik, evidenzgebundene Gluing-Expansion und Boundary-Renewal. Das Genome bestimmt den zulässigen Formraum; Runtimeevidenz bestimmt die momentane Instanziierung; Sedimentation verändert langfristig die Genome-Lineage.

36.3 Navigationserlebnis

Der Benutzer bindet einen Attraktor oder Explorationsmodus. Der Second Layer zeigt Phasenhorizont, lokale Strukturdichte und Tangentenkompass. Physische Bewegung, Blickausrichtung, Sondenwahl und Skalenwechsel werden in denselben Navigationsraum projiziert. Die Maschine hält Welt-, Maschinen- und Humanframe fortlaufend registriert.

36.4 Reference Kernel Handoff

Der nächste Artefakttyp ist ein ausführbarer Reference Kernel mit SchemaBundle, GenomeCompiler, OperatorRegistry, Ledger/Replay und SyntheticWorldAdapter. Dieser Kernel implementiert

R00–R05 und bildet die normative Bodenplatte für alle nachfolgenden Phasen.

Endsatz

GRFPS v3.0.0 ist ein infogenetisch kompiliertes Perzeptionsbetriebssystem: Es erzeugt seine projektiven Wahrnehmungsorgane aus einer versionierten Perzeptionsgrammatik, organisiert sie als kohärenzgetriebenes Operatorfeld, rekonstruiert daraus einen multiskaligen graph- und seamgetragenen Operationsraum, schließt diesen über eine selbstorganisierte Reflexionsgrenze, navigiert ihn attraktorgerichtet und projiziert seine lokale Fortsetzungsgeometrie als Synthetic Second Layer. Eine getrennte Sedimentationslineage kompiliert validierte Laufstruktur in die nächste Genome- und Operatorgeneration.

Part X

Referenzanhänge

Kompakte mathematische Notation

Symbol	Bedeutung
Σ_t	vollständiger Runtimezustand
Γ_P	Perceptual Infogenome
$\mathcal{P}_\Gamma$	kompiliertes Perceptual Program
$\mathcal{O}_i$	zyklisches Projektionsorgan
$\mathcal{W}_i(t)$	gerichtete Exposition / Schwinge
G_t	kohärenzgetriebener Operatorgraph
Γ_t^G	GraphInfogenome
$\mathcal{M}_t$	verklebter multiskaliger Operationsraum
$\mathcal{N}_t$	Nervenkomplex der Patchcover
ρ_P	perzeptive Strukturdichte
$\widehat{\Phi}_t$	modellrelative Spiegelklasse
R_t	selbstadjungierter Reflexionsoperator
$B_t^\pm$	komplementäre Dualphase-Projektoren
Ω_t^*	maximal geschlossene operative Region
$\mathcal{H}_t$	Current/Counterfactual/Open-Horizon
Y_t	Gyrosphere- und Scale-Pose-State
A_e	epochal gebundener Attraktor
v_t	Tangentenkompass mit Scale-Komponente
Λ_n	Sedimentationslineage

Perceptual Genome Schema

```

PerceptualGenome {
  genome_id: Digest,
  version: SemVer,
  lineage: [Digest],
  observables: [ResoniteGene],
  relations: [RelationGene],
  ophan_templates: [OphanGene],
  scale_lattice: ScaleLattice,
  phase_profiles: [PhaseGene],
  attractor_programs: [AttractorGene],
  boundary_programs: [BoundaryGene],
  render_grammar: [RenderGene],
  evolution_programs: [EvolutionGene],
  quantization: QuantizationProfile,
  tolerances: ToleranceProfile,
  budgets: BudgetProfile,
  conformance: ConformanceSuite,
  digest: Digest
}

ResoniteGene {
  id: GeneId,
  input_type: TypeRef,
  output_type: TypeRef,
  operator_ref: Digest,
  quantizer: QuantizerRef,
  scale_scope: [ScaleId],
  phase_scope: [PhaseId],
  witness_contract: WitnessSpec
}

OphanGene {
  id: GeneId,
  aperture: ApertureSpec,
  rotor_profile: RotorProfile,
  projection_profile: ProjectionProfile,
  operator_graph: OperatorGraphRef,
  scheduler_features: FeatureVectorSpec,
  closure_contract: ClosureSpec,
  budgets: BudgetProfile
}

```

}

Rust-orientierte Referenzstruktur

c.1 Crategraph

```
grfps/
  crates/
    grfps-types/           // canonical records and enums
    grfps-canon/           // canonical serialization + digests
    grfps-ledger/          // trace, checkpoints, replay
    grfps-genome/          // AST, normal form, compiler
    grfps-operator/        // registry and typed execution
    grfps-opphan/          // rotor-fiber organs and exposures
    grfps-coherence/        // adaptive operator graph + scheduler
    grfps-infomap/          // graph windows, resonites, gluing
    grfps-mirror/          // mirror fiber + evidence contraction
    grfps-reflection/       // R, B+/B-, boundary region, renewal
    grfps-scale/           // scale lattice and transport
    grfps-gyro/            // orientation, coverage, holonomy
    grfps-nav/             // attractors, horizon, axis, compass
    grfps-view/            // second-layer scene graph
    grfps-effect/          // gated external effect port
    grfps-runtime/         // run state machine
    grfps-sediment/        // slow lineage engine
    grfps-synthworld/      // sealed synthetic reference worlds
  schemas/
  genomes/
  conformance/
  examples/
```

c.2 Deterministischer Kern

Kanonisierung, Schemas, Digests, Genome-Normalform, Operatorauswahl unter festen Seeds, Trace und Replay bilden den deterministischen Kern. Numerische Solver und stochastische Strategien werden durch Seeds, Toleranzprofile, Replayklassen und Resultatquotienten gebunden.

Conformance-Matrix

ID	Witness	Akzeptanzklasse
C00	IdentityWitness	Version, SurfaceDigest und RunDescriptor gebunden
C01	CanonWitness	idempotente Normalisierung und roundtrip-stabile Serialisierung
C02	GenomeWitness	Normalform, Linkage und CompileManifest stabil
C03	OphanWitness	zyklischer Zustand, Exposition und Trace reproduzierbar
C04	SchedulerWitness	Scorebestandteile, Dependencyquotient und Auswahl sichtbar
C05	GraphWitness	Window-Dekomposition und Multi-Signatur reproduzierbar
C06	GlueWitness	Overlap-Kompatibilität, SeamWitness und Nerve stabil
C07	MirrorWitness	Kandidatenfaser und EvidenceDigest konsistent
C08	ReflectionWitness	$R^* = R$, $R^2 = I$, Projektoren und Region zertifiziert
C09	BoundaryWitness	Renewal, Regionlineage und Horizonupdate replaybar
C10	ScaleWitness	Lift/Quotient und Cross-Scale-Roundtrip innerhalb Toleranz
C11	GyroWitness	Coverage, Orientierung, Lift und Holonomie gebunden
C12	AttractorWitness	Epoche, Potential, Stopprädikat und Arrival konsistent
C13	CompassWitness	Richtung/Fächer und Scale-Aktion aus demselben Zustand ableitbar
C14	ViewWitness	jedes Viewobjekt besitzt MachineRef und Provenance
C15	ReplayWitness	deklarierte Replayklasse reproduziert Grenzscheidungen
C16	SedimentationWitness	Candidate zu Genome-Lineage mit vollständigem Delta

C17	BlindWitness	versiegelte Referenzwelt über definierte Baselines ausgewertet
-----	--------------	---

Referenzszenario: Topologische Attraktornavigation

E.1 Setup

Eine synthetische Welt enthält eine verborgene multiskalige Graphstruktur, mehrere kospektrale lokale Patches, einen driftenden Orientierungsframe und zwei Attraktorklassen. Das Genome definiert ResoniteGenes für lokale Invarianten, OphanGenes für zyklische Sonden, drei Skalenstufen und einen Attraktorvertrag.

E.2 Lauf

Der Run bindet Zentrum und Frames, kompiliert das Genome, erzeugt initial vier Ophanim und beginnt mit geringer Strukturdichte. Die ersten GraphResonites bilden isolierte Patches. Der Scheduler erkennt geringe Coverage und instanziiert weitere Ophanim. Neue Overlaps erzeugen SeamWitnesses, der Nervenkomplex wächst und ρ_P steigt. Die Reflection Engine vergrößert Ω_t^* . Der Phasenhorizont materialisiert zwei Richtungsfamilien.

Der Attraktorvertrag bewertet eine der beiden Familien günstiger. Axis Discovery identifiziert eine fortsetzbare Klasse, der Tangentenkompass zeigt einen scharfen Pfeil. Ein ScaleGene empfiehlt anschließend den Übergang von Meso zu Mikro, weil dort die kospektrale Mehrdeutigkeit durch TracePower- und Boundary-Observablen getrennt werden kann. Nach dem Scale-Lift kollabiert der Fächer auf eine eindeutige Richtung.

E.3 Closure

Die Attraktorankunft wird durch Zielprädikate, BoundaryWitness, ScaleRoundtrip, AxisPersistence und Replay bestätigt. Der HumanView zeigt die erreichte Attraktorklasse, die geschlossene lokale Boundary und den vollständigen Tracepfad.

Referenzszenario: selbstorganisierte Reflexionsgrenze

F.1 Setup

Eine Welt wird schrittweise sichtbar. Zu Beginn liegen vier isolierte Patches und ein großes Open-Horizon vor. Das Genome priorisiert BoundaryGain und GraphGlue.

F.2 Expansion

Jede neu instanziierte Sondenfamilie erzeugt lokale GraphWindows. Patches mit kompatiblen Overlaps werden verklebt. Die Optimierung von Ω_t^* bevorzugt eine zusammenhängende, replaystabile Region. Der sichtbare BoundaryRing wächst mit jeder zertifizierten Renewalstufe. Perzeptive Dichte und BoundaryMeasure werden getrennt geführt: ein strukturdichter Bereich kann lokal klein sein, ein strukturarmer Bereich kann geometrisch weit reichen.

F.3 Multiskalige Closure

Auf Makroskala erscheint eine zusammenhängende Region. Ein Scale-Lift zeigt zwei Meso-Unterregionen mit eigener Seamstruktur. Die Cross-Scale-Witnesses binden beide Darstellungen. Der Second Layer rendert Makrogrenze, Meso-Untergrenzen und den Scale-Compass in derselben weltregistrierten Szene.

Perceptual-OS ABI und Systemaufrufe

G.1 ABI-Grundform

Die Perceptual-OS-Oberfläche verbindet DomainAdapter, Genome, Runtime und HumanView über kanonische Aufrufobjekte. Jeder Aufruf besitzt

$$Call = (id, run, epoch, op, input, scope, budget, clock, pre, witness, trace).$$

Die Antwort lautet

$$Return = (call, status, output, residues, witnesses, metrics, post, digest).$$

Damit besitzt jeder Systemaufruf dieselbe Identitäts-, Budget-, Witness- und Replaysemantik.

G.2 Kernaufufe

Aufruf	Eingabe	Ergebnis
Bind	RunDescriptor, Center, Frames, Adapter	WorldBinding und BoundarySeed
LoadGenome	GenomePackage	GenomeState und CompilePlan
Compile	GenomeState, Registry, Adapter	PerceptualProgram
Spawn	OphanTemplate, Context	OphanState
Expose	OphanState, Phase, Aperture	ExposureState
Measure	ExposureState, Evidence	HarmonicDescriptor
Graphify	EvidenceWindow	GraphWindow und GraphResonite
Glue	PatchSet, Overlaps	NerveState und GlueCertificate
Mirror	EvidenceConstraints	MirrorState
Reflect	Map, Mirror, Scope	ReflectionRegion und BoundaryState
Orient	ReflectionRegion, Coverage	GyroState
Target	AttractorSpec	AttractorBinding
Navigate	GyroState, AttractorBinding	NavigationProposal
Render	SealedState, HumanFrame	HumanView
Checkpoint	RuntimeState	CheckpointDigest
Replay	Manifest, Inputs	ReplayResult

Sediment

RunBundleSet, Lineage

GenomeCandidate

G.3 Call Graph

Der Referenzcallgraph ist ein getypter DAG. Seine Spine lautet

$$\begin{aligned} &Bind \rightarrow LoadGenome \rightarrow Compile \rightarrow Spawn \rightarrow Expose \rightarrow Measure \\ &\quad \rightarrow Graphify \rightarrow Glue \rightarrow Mirror \rightarrow Reflect \rightarrow Orient \\ &\quad \rightarrow Navigate \rightarrow Render. \end{aligned}$$

Trace- und Checkpointaufrufe begleiten jede Zustandsänderung. Replay interpretiert denselben Graphen aus kanonischen Inputs und RunDescriptor. Sediment konsumiert versiegelte RunBundles auf der langsamen Lineage-Zeitachse.

G.4 Capability Namespaces

Die ABI führt getrennte Capabilityräume für Observation, Projection, Mapping, Navigation, View und Effect. Ein Operatoraufruf trägt genau den Namespace, den sein Vertrag benötigt. Dadurch kann ein Perceptual Program seine eigenen Wahrnehmungsorgane kompilieren und gleichzeitig eine klar adressierbare Oberfläche für Desktop-, XR- und andere Träger besitzen.

Lifecycle-Automaten des Perzeptionsbetriebssystems

H.1 Genome-Lifecycle

Ein Genome durchläuft

DRAFT → CANONICAL → COMPILED → ACTIVE → SEALED → SEDIMENTED.

DRAFT trägt editierbare Gene. CANONICAL besitzt Normalform und Digest. COMPILED besitzt ein Perceptual Program. ACTIVE bindet eine Runlineage. SEALED besitzt vollständige Evaluations- und Replaywitnesses. SEDIMENTED ist Bestandteil einer kanonischen Lineage.

H.2 Ophan-Lifecycle

Ein Ophan durchläuft

*TEMPLATE → INSTANTIATED → CALIBRATED
→ ACTIVE → QUIESCENT → RETIRED.*

INSTANTIATED bindet Zentrum, Genome und Context. CALIBRATED bindet Apertur, Phase, Projektor und Toleranzen. ACTIVE erzeugt Expositionen. QUIESCENT bewahrt Identität bei ausbleibender Schedulingpriorität. RETIRED fixiert den letzten StateDigest und Trace.

H.3 Map-Lifecycle

Lokale Struktur wächst entlang

*WINDOW → FINGERPRINTED → PATCH → COMPATIBLE
→ GLUED → ATLASSED → SEALED.*

Jede Stufe besitzt einen eindeutigen Witness. Die Map kann mehrere lokale Lineages führen; der Atlas verknüpft diese über Transitionen, Seams und Cross-Scale-Abbildungen.

H.4 Boundary-Lifecycle

Die Reflexionsregion folgt

$$SEED \rightarrow OPEN \rightarrow GROWING \rightarrow STABLE \rightarrow RENEWED.$$

SEED ist die initial gebundene Region. OPEN führt offene Kanten und Horizonklassen. GROWING nimmt neue gluable Patches auf. STABLE erfüllt RegionWitness und BoundaryWitness. RENEWED erzeugt eine neue Boundarylineage mit größerer oder feinerer geschlossener Region.

H.5 Axis-Lifecycle

Die Achse folgt

$$CANDIDATE \rightarrow DISCOVERED \rightarrow LOCKED \\ \rightarrow TRACKED \rightarrow REACQUIRE \rightarrow LOCKED.$$

DISCOVERED besitzt einen SeparationWitness. LOCKED besitzt Attraktorepoche und Persistenzmarge. TRACKED wird durch Paralleltransport plus Evidenzkorrektur fortgesetzt. REACQUIRE erzeugt eine neue Kandidatenfamilie und bindet den nächsten Lock an dieselbe oder eine neue Achsenklasse.

H.6 Sedimentations-Lifecycle

Die langsame Evolution folgt

$$OBSERVE \rightarrow AGGREGATE \rightarrow CANDIDATE \rightarrow ABLATE \\ \rightarrow VALIDATE \rightarrow COMPILE \rightarrow SEDIMENT.$$

Damit wird die Runtime zum Messfeld der eigenen Weiterentwicklung. Jede neue Lineageversion entsteht aus versiegelten, vergleichbaren RunBundles und trägt ihr vollständiges Delta.

Algorithmische Normalformen

1.1 Fast Loop

Der perzeptive Puls besitzt die Referenznormalform:

```
perceptual_pulse(state, event):
    state = ingest_and_bind(state, event)
    need = estimate_discrimination_need(state)
    plan = schedule_ophanim(state, need)
    exposures = expose_active(plan, state)
    evidence = measure_and_canonicalize(exposures)
    patches = graphify(evidence)
    map = update_infogenetic_map(state.map, patches)
    mirror = contract_mirror(state.mirror, evidence, map)
    region = renew_reflection_region(state.region, map, mirror)
    gyro = relax_orientation(state.gyro, region, state.target)
    nav = compile_navigation(gyro, region, state.target)
    view = render_second_layer(nav, region, state.human_frame)
    return seal_trace(state, map, mirror, region, gyro, nav, view)
```

1.2 Ophan-Scheduling

Die Schedulingentscheidung maximiert eine deklarierte Utility unter Budget:

$$o_t^* = \arg \max_{o \in \mathcal{O}_{cand}} (\alpha IG_o + \beta CG_o + \gamma BG_o + \eta TG_o - \lambda Cost_o - \mu Risk_o - \nu Red_o).$$

IG misst Diskriminationsgewinn, CG Coveragegewinn, BG Boundarygewinn, TG Attraktorgewinn und Red dependency-adjustierte Redundanz. Kandidaten können aktivierte Templates, Reorientierungen bestehender Ophanim oder neue kontextgebundene Instanzen sein.

1.3 Graph-Infogenetic Update

```
graph_update(map, evidence):
    windows = decompose(evidence)
```

```

for window in windows:
    canon = canonicalize(window)
    fp     = multi_invariant_fingerprint(canon)
    gene   = emit_graph_resonite(fp, provenance(window))
    map.graph_infogenome.insert(gene)
overlaps = discover_overlaps(map.graph_infogenome)
seams     = witness_compatibility(overlaps)
map.nerve = glue_compatible(map.nerve, seams)
return update_density_and_lineage(map)

```

1.4 Boundary Renewal

```

renew_reflection_region(region, map, mirror):
    candidates = adjacent_gluable_regions(region, map)
    scored     = evaluate_region_objective(candidates, mirror)
    selected   = choose_max_closed(scored)
    witness    = certify_reflection(selected)
    return commit_region(selected, witness)

```

1.5 Slow Loop

```

sedimentation_cycle(lineage, sealed_runs):
    corpus = aggregate_run_bundles(sealed_runs)
    motifs = extract_stable_operator_motifs(corpus)
    candidates = compile_genome_deltas(motifs, lineage)
    evaluated  = run_ablations_and_blind_tests(candidates)
    winner     = select_surface_improving_candidate(evaluated)
    return sediment_new_lineage(lineage, winner)

```

1.6 Dualzeitige Konsistenz

Fast Loop und Slow Loop kommunizieren über versiegelte RunBundles und versionierte GenomePackages. Ein aktiver Run bleibt an seine GenomeDigest gebunden. Die nächste Lineage wird in einer neuen Runserie aktiviert. Dadurch bleiben perzeptive Echtzeit und strukturelle Evolution getrennt adressierbar und gemeinsam replayfähig.

Multiskalige Faserung und Skalenkompass

J.1 Skalenposet

Die Skalenstruktur ist ein endlicher oder effektiv budgetierter Poset

$$(\mathcal{S}, \preceq) = \{s_0, s_1, \dots, s_k\}.$$

Jede Skala besitzt einen lokalen Operationsraum $\mathcal{M}^{(s)}$, ein GraphInfogenome $\Gamma_G^{(s)}$, einen Horizon $\mathcal{H}^{(s)}$ und eine ReflectionRegion $\Omega^{\star, (s)}$.

J.2 Lift und Quotient

Für $s \preceq s'$ existieren getypte Abbildungen

$$L_{s \rightarrow s'} : \mathcal{M}^{(s)} \rightarrow \mathcal{M}^{(s')}, \quad Q_{s' \rightarrow s} : \mathcal{M}^{(s')} \rightarrow \mathcal{M}^{(s)}.$$

Ein Cross-Scale-Witness prüft

$$Q_{s' \rightarrow s}(L_{s \rightarrow s'}(x)) \sim_{\epsilon_s} x.$$

Die Äquivalenzklasse erlaubt feine Strukturen, deren Makroprojektion identisch bleibt.

J.3 Skalenresonanz

Ein Attraktor kann über mehrere Skalen verteilt sein. Seine Skalenresonanz wird durch

$$R_A(s) = \text{Support}(A \mid s) \cdot \text{Identiifiability}(A \mid s) \cdot \text{BoundaryGain}(A \mid s)$$

modelliert. Der Navigator kann dadurch eine laterale Fortsetzung oder eine Skalenverschiebung empfehlen.

J.4 Tangentenkompass im Produktbund

Der lokale Navigationsvektor lautet

$$v_t = (v_t^{\parallel}, v_t^{\perp}) \in T_{x_t} \mathcal{M}^{(s_t)} \oplus T_{s_t} \mathcal{S}.$$

$v^{\parallel}$ ist die Fortsetzungsrichtung im aktuellen Chart. Die Scale-Komponente wählt

$$v^{\perp} \in \{\text{HOLD}, \text{LIFT}, \text{QUOTIENT}\}.$$

Die HumanView kann beide Komponenten als Richtungsmarker plus Tiefen-/Zoomindikator projizieren.

J.5 Scale Atlas

Der rollende Atlas besitzt zusätzlich zu Charttransitionen einen ScaleGraph. Eine Atlaszelle ist

$$a = (\text{chart}, \text{scale}, \text{parent}, \text{children}, \text{lifts}, \text{quotients}, \text{seams}, \text{holonomy}, \text{trace}).$$

Damit kann eine lokale Struktur sowohl räumlich/topologisch als auch über Auflösungsebenen fortgesetzt werden.

J.6 Beispiel einer dreistufigen Leiter

Eine Referenzinstanz nutzt $\mathcal{S} = \{\text{micro}, \text{meso}, \text{macro}\}$. Makro führt globale Attraktorregionen, Meso führt Subsysteme und Seams, Mikro führt lokale Graphfenster und feine Observablen. Ein Makrokompass kann einen Meso-Lift anfordern; der Meso-Navigator kann zur Disambiguierung einen Mikro-Lift erzeugen; nach Closure wird das Ergebnis über Quotienten wieder in Meso und Makro sedimentiert.

Second-Layer Rendergrammatik

κ.1 Viewobjekt

Jedes sichtbare Objekt ist eine Projektion eines versiegelten Maschinenrecords:

$$VObj = (id, machineRef, geometry, style, facticity, uncertainty, horizon, scale, interaction, provenance).$$

Die Rendergrammatik ist ein Genomebestandteil und kann für Desktop, XR oder andere Träger kompiliert werden.

κ.2 Kanonische Visual Classes

Klasse	Maschinenbedeutung	Referenzprojektion
AttractorMarker	gebundene Zielklasse	räumlicher Fixpunkt / Rahmen / Region
TangentArrow	eindeutige lokale Fortsetzung	gerichteter Pfeil
DirectionFan	mehrere fortsetzbare Klassen	Fächer / Mehrfachvektor
HorizonRing	aktuelle Reflexionsgrenze	Ring / Kuppel / Kontur
OpenRegion	noch offene Struktur	transparente oder diffuse Zone
PatchSurface	kompatibel verklebte lokale Struktur	Fläche / Netz / Zellfeld
SeamLine	bezeugte Transition	Naht / Kante / Übergangsband
ScalePortal	Lift-/Quotientenaktion	Tiefenmarker / Skalenring
ResidueGlyph	typisierte offene Pflicht	lokales Residuumssymbol
TraceTrail	historische Fortsetzung	Spur / Verlaufslinie
AxisFrame	verriegelte Achsenklasse	lokaler Orientierungsrahmen
DensityField	ρ_P -Verteilung	Opazität / Detailgrad / Geometriedichte

K.3 Dichte als Materialisierung

Die Renderfunktion koppelt Strukturdichte an Detailgrad:

$$LOD(x) = Q_{lod}(\rho_P(x), U(x), scale(x), budget).$$

Niedrige Dichte erzeugt sparse Marker. Mittlere Dichte erzeugt Kanten, Seams und Flächen. Hohe Dichte erzeugt gebundene lokale Architektur. Das Resultat bleibt durch LOD-Wechsel auf dieselben MachineRefs rückführbar.

K.4 Phasenhorizont als Panorama

Der Horizon wird als dreifache Viewmenge gerendert:

$$V_H = V_{current} \cup V_{counter} \cup V_{open}.$$

Current trägt gegenwärtig fortsetzbare Richtungen. Counter trägt modellierte alternative Fortsetzungen. Open trägt Regionen, deren Struktur durch zusätzliche Ophanim, Evidenz oder Skalenwechsel auflösbar wird.

K.5 Interaktion

Der Minimalmodus besitzt vier Interaktionen: **SelectAttractor**, **Align**, **Forward** und **Scale**. Komplexere Profile können Probeanforderungen, Replaynavigation, Patchinspektion und Horizonfilter ergänzen. Jede Interaktion erzeugt ein typisiertes InputEvent und damit einen reproduzierbaren Runtimepfad.

Ledger, Content Addressing und perzeptives Gedächtnis

L.1 Adressmodell

Jedes kanonische Runtimeartefakt erhält

$$id = H(type || version || canon(payload) || parentDigests).$$

Der Digest verbindet Typ, Version, kanonischen Payload und Lineage. Das Ledger bildet daraus einen gerichteten Evidenz- und Transformationsgraphen.

L.2 TraceSegment

Ein TraceSegment lautet

$$T = (run, parent, input, operator, pre, post, witnesses, residues, clocks, detClass, digest).$$

TraceSegments werden append-only verkettet. Checkpoints verdichten eine Präfixmenge zu einem adressierbaren StateDigest und bewahren die Referenzen auf die zugrunde liegenden Segmente.

L.3 Perzeptives Gedächtnis

Das Gedächtnis besitzt vier Ebenen:

1. **Event Memory:** rohe gebundene Ereignisse und EvidenceRefs;
2. **Structural Memory:** GraphResonites, Seams, Atlas und Boundarylineage;
3. **Procedural Memory:** Genome, Operatorprogramme, Schedulerprofile;
4. **Replay Memory:** RunDescriptor, Seeds, Checkpoints, Digests und ReplayManifest.

Diese Ebenen teilen dieselbe Content-Addressing-Semantik und können separat persistiert oder streamend nachgeladen werden.

L.4 Snapshot und Rolling State

Der Livezustand ist ein Rolling Snapshot über Ledgerrefs. Ein Snapshot trägt die minimalen aktiven IDs, während große Evidenzmengen im adressierten DAG verbleiben. Dadurch kann die Runtime einen kompakten Hot State führen und gleichzeitig vollständige Provenienz besitzen.

L.5 Reanalyse

Ein versiegelter Run kann mit einem neuen Genome oder einer neuen algorithmischen Profilversion erneut ausgewertet werden. Die Reanalyse erzeugt eine neue Lineage und referenziert dieselbe Evidenzbasis. Unterschiede werden als Delta zwischen den resultierenden kanonischen Klassen ausgewiesen.

Synthetic-World Benchmark Suite

M.1 Harnessprinzip

Die Referenzwelt trennt Generator und Evaluator. Der Generator besitzt eine verborgene Ground Truth, erzeugt beobachtbare Events und beantwortet Ophanprobes. Die Runtime erhält ausschließlich den DomainAdapter-Port. Der Evaluator öffnet Ground Truth nach Seal und berechnet Metriken.

M.2 Phantomfamilien

Die Basissuite enthält 24 Welttypen:

ID	Familie	Kernwitness
W01	Sparse Ring	zyklische Phase und Closure
W02	Broken Ring	offene Seam und Horizon
W03	Dual Sheet	Dualphase und Projektortrennung
W04	Triple Sheet	mehrblättrige Phasenführung
W05	Rotor Sphere	gyrosphärische Coverage
W06	Occluded Lattice	OpenRegion und Reacquisition
W07	Cospectral Twins	Multi-Signatur und Holdklasse
W08	Isomorphic Relabel	kanonische Graphidentität
W09	Scale Alias	Cross-Scale-Witness
W10	Nested Attractors	Attraktorepoche und ScaleCompass
W11	Moving Attractor	Axis Track und Transport
W12	Target Switch	Epochenisolation
W13	Environment Drift	Reanchor und Reacquire
W14	Seam Conflict	Residualisierung und Patchsplit
W15	False Overlap	GluingCertificate
W16	Hidden Bridge	aktive Sondengenerierung
W17	Redundant Sensor Bank	Dependencyquotient
W18	Complementary Sensors	Synergiegewinn
W19	Boundary Growth	ReflectionRegion Renewal
W20	Boundary Contraction	Regionlineage und Horizonupdate
W21	Holonomy Loop	Transport- und Closurewitness

W22	Counterfactual Fork	DirectionFan und Counter-Horizon
W23	Dense Core	Strukturdichte und LOD
W24	Mixed Multiscale World	End-to-End Perceptual-OS-Witness

M.3 Metriksatz

Die Basismetriken sind

$$M = (Acc_{class}, Cal, IG, Rank_{eff}, GluePrec, GlueRec, BoundaryIoU, \\ AxisErr, NavRegret, ScaleRegret, ReplayRate, Latency, Cost).$$

BoundaryIoU vergleicht geschätzte und Ground-Truth-Region im zulässigen Scope. *AxisErr* misst Winkel- oder Klassenfehler der Fortsetzungsachse. *NavRegret* vergleicht die gewählte Fortsetzung mit der besten zulässigen Ground-Truth-Fortsetzung. *ScaleRegret* bewertet verspätete oder unnötige Skalenwechsel.

M.4 Ablationsfamilien

Jede Weltsuite wird mit vollständigem System und gezielten Ablationen ausgeführt: feste Ophanbank, deaktivierter Dependencyquotient, Einzel-Skala, statische Boundary, starrer Scheduler, Atlas mit einfachen Witnesses, Renderer mit konstanter Dichte und Slow Loop mit eingefrorenem Genome. Der Effekt jeder Struktur wird dadurch als messbares Delta sichtbar.

M.5 Blind Integration Witness

W24 bildet den ersten großen Integrationswitness. Es verlangt GenomeCompile, adaptive Ophanbank, GraphInfogenome, Gluing, Mirror, ReflectionRegion, Multiscale, Gyrosphere, Attraktor, Tangentenkompass, HumanView und Replay in einem versiegelten Lauf. Der Evaluator akzeptiert den Run, wenn die vorab festgelegten Metrikschwellen und Reproduktionsklassen erreicht werden.

XR Carrier Contract und physische Träger-schnittstelle

N.1 Carrier Interface

Die Trägerschnittstelle wird als Adaptervertrag

$$CI = (Pose, Frames, Clock, Input, Display, Sensors, Occlusion, Calibration, Latency, Capabilities).$$

geführt. Der Maschinenraum arbeitet gegen diesen Vertrag; ein konkretes Headset instanziiert ihn.

N.2 Framegraph

Der physische Adapter führt mindestens

$$F_W \leftrightarrow F_M \leftrightarrow F_H \leftrightarrow F_D,$$

mit Weltframe, Maschinenframe, Humanframe und Displayframe. Jede Transformation trägt Timestamp, CalibrationRef, Covariance/Uncertainty und TraceRef.

N.3 Pose und Weltregistrierung

Die Runtime erhält eine zeitindexierte Pose

$$p_t = (x_t, q_t, \Sigma_t^{pose}, clock).$$

Der ViewCompositor transformiert MachineGeometry über den aktuellen Framegraph in Display-Geometry. Stale-Grenzen, Occlusionstatus und Latenzprognose werden als explizite Viewattribute geführt.

N.4 Carrier-Simulation

Vor dem physischen Prototyp emuliert der Desktop-Harness Pose, Trackingverlust, Clockdrift, Occlusion, variable Sichtfelder und Eingabelatenz. Dieselbe CI-Schnittstelle wird anschließend an den realen Träger gebunden. Damit besitzt die Hardwarephase einen bereits geschlossenen Softwarevertrag.

N.5 Minimaler XR-Prototyp

Der erste XR-Witness rendert Zentrum, HorizonRing, PatchSurface, AttractorMarker, TangentArrow/DirectionFan, ScalePortal und TraceTrail. Nutzerinteraktionen umfassen TargetSelect, Align, Forward, Hold und Scale. Der Prototyp führt denselben ReplayManifest wie der virtuelle Harness und misst FrameRegistrationError, DisplayLatency, ViewTraceability und InteractionRoundtrip.

N.6 Capability Envelope

Nach Messung des physischen Trägers wird

$$Cap = (domains, scales, latency, accuracy, coverage, \\ robustness, interaction, resourceEnvelope).$$

als Capability Envelope versiegelt. Es bindet die reale Betriebsreichweite der konkreten Instanz und wird als Adapterprofil im GenomePackage referenziert.

Glossar

Begriff	Kanonische Bedeutung
Perceptual Operating Substrate	Runtime, Compiler, Karte, Boundary, Navigation, View und Lineage in einer typisierten Maschinenidentität
Perceptual Infogenome	ausführbare Perzeptionsgrammatik einer Instanz
Ophan	zyklisches, kompilierbares Projektionsorgan
Schwinge	momentane gerichtete Exposition eines Ophans
Harmonic Field	gewichtete Feldsicht aktiver Schwingen und Deskriptoren
GraphResonite	provenienzgebundene lokale Graphsignatur
GraphInfogenome	kanonische Menge/Sequenz lokaler Strukturgene
Nervenkomplex	Simplizialer Komplex bezeugter Patchüberlappungen
Mirror-Faser	Menge der mit aktueller Evidenz kompatiblen Modellzustände
Reflexionsgrenze	selbstadjungierte, closure-zertifizierte Grenze der operativen Region
Phasenhorizont	Current-, Counterfactual- und Open-Fortsetzungsraum
Strukturdichte	lokales Maß gebundener, kompatibler, evidenzgetragener Struktur
Gyrosphäre	dynamischer Orientierungs- und Coverage-Regulator
Tangentenkompass	lokale Fortsetzungsrichtung plus Skalenaktion relativ zum Attraktor
Sedimentation	langsame Kompilation validierter Laufstruktur in neue Genome-/Operatorlineage

Literaturanschluss

Bibliography

- [1] R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8), 1988.
- [2] R. Bajcsy, Y. Aloimonos, J. Tsotsos. Revisiting active perception. *Autonomous Robots*, 2018.
- [3] L. P. Kaelbling, M. L. Littman, A. R. Cassandra. Planning and acting in partially observable stochastic domains. *Artificial Intelligence*, 101, 1998.
- [4] S. Thrun, W. Burgard, D. Fox. *Probabilistic Robotics*. MIT Press, 2005.
- [5] F. Dellaert, M. Kaess. Factor graphs for robot perception. *Foundations and Trends in Robotics*, 2017.
- [6] A. Hatcher. *Algebraic Topology*. Cambridge University Press, 2002.
- [7] S. Mac Lane, I. Moerdijk. *Sheaves in Geometry and Logic*. Springer, 1992.
- [8] H. Edelsbrunner, J. Harer. *Computational Topology*. AMS, 2010.
- [9] G. Carlsson. Topology and data. *Bulletin of the AMS*, 46, 2009.
- [10] K. Shoemake. Animating rotation with quaternion curves. *Computer Graphics*, 19(3), 1985.
- [11] R. Hermann, A. Krener. Nonlinear controllability and observability. *IEEE TAC*, 22(5), 1977.
- [12] D. Q. Mayne et al. Constrained model predictive control: stability and optimality. *Automatica*, 36(6), 2000.
- [13] J. H. Saltzer, D. P. Reed, D. D. Clark. End-to-end arguments in system design. *ACM TOCS*, 2(4), 1984.

Kanonischer Release-Hinweis

GRFPS v3.0.0 trägt die vollständige Perceptual-OS-Architektur dieser Revision. Der nächste maschinennahe Artefaktzustand ist der Reference Kernel R00–R05 mit kanonischen Schemas, Perceptual Genome Compiler, Operatorregistry, Trace/Replay und SyntheticWorldAdapter.

GRFPS v3.0.0

**PERCEPTUAL OPERATING SUBSTRATE FIXPOINT
REFERENCE KERNEL RELEASED FOR IMPLEMENTATION**